CAMARINES SUR POLYTECHNIC COLLEGES
ISO 9001:2015 Certified

CAMARINES SUR POLYTECHNIC COLLEGES
COLLEGE of COMPUTER STUDIES

THEMESYNC: A GRAPH-BASED DESIGN ORCHESTRATION FRAMEWORK WITH REAL-TIME CONSTRAINT SATISFACTION AND STATEFUL REFINEMENT

A Thesis Project
presented to the Faculty of
College of Computer Studies
Camarines Sur Polytechnic Colleges

In Partial Fulfillment of the Requirements
for the degree Bachelor of Science in Computer Science

By
Guiller Angelo Hermoso
Rona Mae Quite
Michelle Tercero

October 2026

CHAPTER 1

INTRODUCTION

This chapter served as an introduction to the research topic and highlighted why it was relevant. This chapter provided the study's background by discussing and identifying the research problems and outlining the research objectives and questions. It also highlighted the study's significance, scope, and limitations. Furthermore, this chapter included a project dictionary defining key terms and concepts used throughout the study.

Background of the Problem

In the fast changing digital landscape, the creative industry is seeing an unprecedented increase in demand for automated design solutions. The market for AI-powered design tools is expected to increase from $5.54 billion in 2024 to $6.77 billion in 2025, representing a compound annual growth rate (CAGR) of more than 22% [17]. The growth is driven by the requirement for rapid asset development and cross-platform consistency, which aligns with UN Sustainable Development Goal 9, which stresses innovation, and Goal 12, which focuses on minimizing redundant design labor. Maintaining visual brand consistency, such as logos, color palettes, and typography, is no longer an aesthetic choice for businesses but a strategic imperative for establishing consumer trust and loyalty.

Historically, design tools have transitioned from manual, coordinate-based software like Adobe Photoshop to more accessible, template-driven platforms like Canva. This progression toward intelligent design is further exemplified by emerging industry research such as Pomelli by Google Labs. Pomelli utilizes semantic modeling to map high-level aesthetic descriptions directly to component styles, allowing for a more intuitive translation of creative intent [7]. However, a significant "Generative Gap" remains. Current industrial tools

often treat designs as stochastic or static entities. When a design is exported, it is typically ”flattened” into pixel-based formats (JPEG/PNG), which effectively destroys the underlying ”Visual DNA”—the mathematical and relational logic of the elements. This leads to Destructive State Management, where any manual refinement made by a user is lost the moment a global theme update is applied.

Furthermore, existing tools frequently suffer from Layout Fragility across Heterogeneous Dimensions. Many industrial tools rely on absolute coordinate-based layouts that lack relational logic. When a user transitions from a vertical format (like an ID) to a horizontal one (like a certificate), the system often fails to determine optimal placement, leading to Responsive Layout Failures (RLFs). These failures include elements protruding off edges or overlapping as layout space becomes scarce, making the output look unprofessional or non-functional [1, 21].

Also, while modern Generative AI models can produce high-fidelity imagery, they are often stochastic and computationally ”opaque”. These models can ”hallucinate” artistic elements but lack the Deterministic Vector Structures (SVG) necessary for precise, rule-based orchestration. There is a critical need for a system that treats a design not as a flat file, but as a Stateful Collection of Objects that can maintain visual harmony through mathematical constraints.

Within the context of academic and organizational environments, the need for consistent branding across different common publication materials is a recurring challenge. Without a framework that utilizes Graph Neural Networks (GNN) to understand structural hierarchies and Constraint Satisfaction Problem (CSP) Solvers to maintain layout requirements, producing these assets remains a labor-intensive process prone to human error. Without a shared relational model that encodes an organization’s stylistic and structural constraints, even minor brand updates — such as a color refinement or a logo revision — cascade into a full manual redesign cycle across every affected publication material. This gap highlights the pressing need for an automated, constraint-aware approach to brand consistency.

ThemeSync addresses these gaps by proposing a graph-based orchestration framework. By utilizing K-Means Clustering for color extraction and the Scale-Invariant Feature Transform (SIFT) for keypoint analysis, the system can autonomously identify a brand's stylistic markers. By integrating a Bi-directional State Machine, the framework ensures that manual user "nudges" are preserved as "Hard Constraints," allowing for a seamless "Human-in-the-Loop" experience that balances automated efficiency with creative control. By treating every design asset as a node within a relational graph, brand updates reach every format initialized from the master layout.

Statement of the Problem

The rapid transition to multi-platform digital branding demands tools that can seamlessly maintain visual integrity across heterogeneous formats. However, current automated design systems rely heavily on static templates or stochastic generation, creating a significant "Generative Gap" where outputs are rendered as computationally opaque, flat files (e.g., JPEG or PNG). This limitation leads to destructive state management, meaning any manual refinement or "nudge" made by a user is mathematically irreconcilable with the master theme and is subsequently lost during global updates. Furthermore, because these industrial tools utilize absolute coordinate-based layouts rather than relational logic, they suffer from layout fragility; transitioning a design between vastly different aspect ratios frequently results in broken layouts that require extensive, labor-intensive manual correction.

To address these computational bottlenecks, this study developed ThemeSync, a specialized design orchestration framework that transitioned away from immutable files by treating layouts as a stateful collection of objects. The developed method utilized a Constraint Satisfaction Problem (CSP) Solver, specifically the Cassowary Algorithm, to treat manual user edits as "Hard Constraints," triggering real-time recalculations to maintain vi-

sual harmony among surrounding elements. Coupled with Graph Neural Networks (GNN) to map structural dependencies and a Bi-directional State Machine to serialize data, the system ensured that local human-in-the-loop refinements and typography updates were synchronized without data loss.

Ultimately, the purpose of this research was to develop and evaluate this orchestration framework to bridge the gap between automated synthesis and manual refinement. By doing so, the study aimed to answer how Graph-Based DNA Extraction could successfully preserve stylistic markers across varying formats, and to what extent a CSP solver could effectively manage real-time manual edits while maintaining global design rules.

Objectives of the Study

General Objective

This study aims to design and evaluate ThemeSync, a graph-based design orchestration framework that implements real-time constraint satisfaction and stateful manual refinements across heterogeneous design formats

Specific Objectives

To achieve the general objective, the researchers aim to accomplish the following:

1. To design a Relational Logic and Feature Extraction model integrating Graph Neural Networks (GNN), K-Means Clustering, and the SIFT algorithm to autonomously identify and map structural dependencies and stylistic markers from master design files.

2. To develop a Reactive Layout Synthesis engine utilizing a Bi-directional State Machine and the Cassowary Algorithm to solve Constraint Satisfaction Problems (CSP),

enabling the real-time propagation of manual ”Hard Constraints” while preserving global layout states.

3. To evaluate the architectural integrity and professional viability of the generated multi-format layouts by implementing a mathematically calibrated Heuristic Fitness Function to quantitatively rank and optimize the aesthetic validity of parallel design candidates, followed by a quantitative human-expert validation.

Significance of the Study

This study aimed to enhance the traditional design workflow by integrating a graph-based orchestration framework designed to improve the efficiency of producing layouts and to reduce manual labor, and that kept every element within its constraints across formats. This study was intended to benefit the following:

Students and Faculty Designers. Designing school-wide materials such as IDs, certificates, and posters often becomes inefficient due to repetitive resizing and layout inconsistencies. A system for creating ”Master Themes” that automatically readjusted themselves to the appropriate size reduced the need for manual formatting of each file, allowing students and faculty to concentrate on subject matter and creative aspects rather than on avoiding technical errors.

Camarines Sur Polytechnic Colleges. The college could benefit from a more efficient process for producing school materials. Automating design tasks would allow the institution to save time and effort on large projects while keeping official documents consistent, accurate, and professional, reducing the operational burden on staff responsible for managing and producing institutional publications.

Creative Professionals and Industrial Organization. Current design tools and production workflows often require manual, repetitive adjustments when resizing assets for various platforms, leading to ”Layout Fragility” and high labor costs. This study provided

a specialized pipeline for Multi-Format Asset Orchestration that utilized a Constraint Satisfaction Problem (CSP) Solver to automate the scaling of ”Visual DNA.”

Researchers. The study contributed to the fields of Intelligent Systems and Design Automation by exploring the intersection of Graph Neural Networks (GNN) and Deterministic State Orchestration. It provided a methodology for Object Serialization and Bi-directional State Machines, offering insights into how design intent—traditionally a subjective human trait—could be mathematically preserved and synchronized within a digital framework.

Future Researchers. The system established a technical foundation that allows future scholars to enhance the automation engine or adapt its algorithms for other digital media. Developers and scholars can utilize the study's findings on Heuristic Search and Aesthetic Scoring, including the experts' preference for evenly spaced layouts, to build more advanced, user-centric technologies or to investigate real-time collaborative design environments.

Computing Field. The study applied Artificial Intelligence (AI) and Computer Vision by utilizing K-Means Clustering for color extraction and Heuristic Search Algorithms for layout suggestions. It demonstrated how a Human-in-the-Loop (HITL) approach could combine mathematical precision with human creativity, and it offered an approach to the ”Generative Gap” by converting flat, ”opaque” files into Stateful Vector Structures, thereby extending the capabilities of smart design environments.

Scope and Limitation

The primary scope of this research involved the design, development, and technical evaluation of ThemeSync, a graph-based design orchestration framework engineered to automate brand identity synchronization across heterogeneous formats while addressing the challenges of “Generative Gap” and “Layout Fragility” in conventional design tools. Specifically, the study sought to design a Relational Logic and Feature Extraction model

that integrated Graph Neural Networks, K-Means Clustering, and the SIFT algorithm to identify and map structural dependencies and stylistic markers from master design files. Furthermore, the scope encompassed the implementation of Dynamic Node Instantiation, enabling the framework to merge extracted ”Visual DNA” with external administrative datasets—like user-defined JSON payloads containing specific names, titles, or signature counts—to generate fully populated, variable-driven documents. It also aimed to implement a Reactive Layout Synthesis engine powered by a Bi-directional State Machine and the Cassowary Algorithm to solve Constraint Satisfaction Problems, ensuring real-time propagation of manual “Hard Constraints” and preservation of global layout states during resizing or format transitions. Lastly, the research intended to evaluate the architectural integrity and aesthetic quality of generated multi-format layouts through heuristic search and a mathematically calibrated fitness function, thereby optimizing structural consistency and visual validity across diverse platforms.

The study was limited to the synchronization of 2D graphical layouts and branding assets; it did not extend to 3D modeling or video-based motion graphics. Furthermore, while the system would provide ”Aesthetic-Aware” suggestions, the final design authority remained with the human user, focusing on a Human-in-the-Loop refinement process rather than autonomous design replacement. Consequently, while the core computational evaluation of ThemeSync relied on rigorous mathematical benchmarking—specifically measuring algorithmic latency, layout detection accuracy via Intersection over Union (IoU), and constraint satisfaction rates—the validation of the final generated layouts did not rely exclusively on theoretical metrics. To ensure that the framework’s outputs aligned with real-world professional standards, the study integrated a mixed-methods evaluation strategy. The validated design candidates were then evaluated by a panel of four domain experts (professional layout artists and graphic designers with documented industry experience) using a quantifiable metric. This approach was designed to verify that the framework satisfied both computational constraints and practical industry aesthetic requirements.

Project Dictionary

This section provides the definitions of the major terms and concepts that are used throughout this study, serving as a guide in understanding the technologies, methods, and components involved in developing ThemeSync.

Affine Transformations. An affine transformation is the group of transformations that leave the parallelism of straight lines fixed, it can be combined with any transformation consisting of rotation, translation, shearing, and scaling [18]. In this study, Affine Transformations are mathematical functions that are used to transform coordinates and hierarchies of visual images between the master design and new canvas sizes in heterogeneous aspect ratios.

Bi-Directional state machine. A state machine is a form of automation where the execution of the system operates with a finite amount of states and transitions to another state based on an activity or condition [19]. In this study, the calculation algorithm is the one that is used to coordinate the refinements made manually with the automated layouts by sequentializing the refinements into a stable JSON state.

Cassowary algorithm. Cassowary is an incremental constraint-solving algorithm that efficiently solves systems of linear equalities and inequalities, in which each constraint is either a requirement or a weighted preference; it was developed for user interface layout [2]. In this study, this algorithm is the main layout engine that re-calculates soft constraints when it is injected with manual Hard Constraints to ensure that design elements are not overlapping.

Constraint Satisfaction Problem (CSP). Constraint Satisfaction Problem (CSP) are core in artificial intelligence and theoretical computer science, a model of variables with domains in finite sets and a list of constraints, which are to be satisfied [6]. In this study, the mathematical model used to control layout density and support recalculation of design

nodes in real time to create visual harmony is CSP. This enables the system to dynamically enforce branding constraints across all generated publication materials.

Dynamic Node Instantiation. Dynamic Node Instantiation is the algorithmic process by which external, user-defined variables are assigned baseline geometric dimensions and converted into functional mathematical nodes. The JSON format acts as a critical standard for lightweight data exchange, and its nested structure is uniquely suited for representing these diverse UI properties [3]. In this study, Dynamic Node Instantiation enables seamless communication between the frontend interfaces and the backend Graph Neural Network, allowing dynamic data to be orchestrated alongside the master theme.

Fitness Function. This is a quantitative figure of merit used to evaluate how effectively a candidate solution satisfies established design composition features and structural requirements [8]. In this study, the mathematical equation to rank the layout candidates is the Fitness Function, which is used to determine how well the layouts meet design rules such as alignment to a structural grid and the ratio of whitespace to other elements.

Graph Neural Network (GNN). Graph Neural Network (GNN) is a type of neural network that represents graph dependencies by exchanging messages among graph nodes in order to address problems that involve non-Euclidean space [4]. In this study, the GNN is the machine learning model that is applied to analyze design hierarchies where elements are viewed as nodes and the relationship between elements as edges to determine how best to position elements.

Hard Constraint. Hard Constraint is a requirement that must be strictly satisfied in a constraint optimization problem [20]. In this study, any manual user edit, such as moving or resizing an element, is registered as a “Hard Constraint” that the system must respect while adjusting other elements.

Implicit Geometric Hierarchy. Implicit Geometric Hierarchy is the mathematical principle by which the framework establishes visual priority. Graph Neural Networks are capable of capturing irregular object boundaries and complex spatial relationships with

high accuracy by treating distinct areas or patches as individual graph nodes [9, 12]. In this study, rather than relying on explicit semantic tags, visual priority is encoded in each element's baseline geometric dimensions and reading order: the Spatial Hierarchy Index sets each element's role and its position in the reading order, and each role is assigned a fixed baseline bounding box, so that the title receives the largest box and the recipient's or attendee's name one of the largest.

K-Means clustering. A basic activity within data analysis is clustering, which sets out to arrange a sizeable dataset into significant subsets along the similarity measures, with the K-Means algorithm being modest in the sense that the algorithm ascertains the data points and assigns them to the closest cluster centroid [13]. In this study, K-Means Clustering is the algorithm that is utilized in order to automatically identify dominant and accent color palettes in the pixels of a Master Design file.

Mean Opinion Score (MOS). A standardized, quantitative metric originally formulated to assess the subjective quality and perceptual fidelity of system outputs through structured human evaluation [10]. In this study, MOS refers to the structured five-point rating scale rubric utilized by domain experts to quantitatively grade the algorithmically generated multi-format layouts across three formalized evaluation parameters: Visual Hierarchy Preservation, Brand and Typographic Consistency, and Constraint and Whitespace Harmony.

Object Serialization. Object Serialization is the process of transforming complex data structures into a standardized format to enable real-time synchronization and system architecture persistence [16]. In this study, Object Serialization refers to the transformation of visual design objects into a structured JSON format to preserve a "Live State Link" so that these objects can be post-generatively editable.

Semantic Layout Parsing. Semantic Layout Parsing is the process of identifying and categorizing different parts of a visual layout to understand its underlying structure [5]. In

this study, it is a pipeline used for PNG inputs that utilizes object detection (YOLO) and OCR to "unflatten" raster images into editable semantic nodes.

SIFT (Scale-Invariant Feature Transform). SIFT is an algorithm in computer vision used to detect and describe local features in images [14]. In this study, SIFT keypoint density is used to rank the decorative motifs extracted from the master design and to build the saliency map against which the legibility of the generated text is scored.

Soft Constraints. Soft Constraints are preferences or goals within a layout system that are not strictly required for a valid solution but are optimized to achieve the most desirable aesthetic or functional outcome [11]. According to Jiang, Stuerzlinger, Zwicker, and Lutteroth (2020), these constraints allow for flexible user interface layouts that can adapt to varying screen dimensions while maintaining proportional harmony. In this study, Soft Constraints represent the initial automated placement of design elements which can be overriden or re-calculated when a user introduces manual Hard Constraints.

Visual DNA. An algorithm to encode images as distributions of neuron activations (DNAs) to form an expressive, high-dimensional blueprint enabling visual attributes in one dataset to be compared and preserved in another dataset [15]. In this study, Visual DNA refers to a hybrid data structure combining autonomously extracted algorithmic features (color centroids and relational logic) with user-defined administrative constraints (specific font families, weights, and baseline sizes).

Notes

[1] Ibrahim Althomali, Gregory M. Kapfhammer, and Phil McMinn. 2022. Automated repair of responsive web page layouts. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 140–150. DOI: 10.1109/icst53961.2022.00025.

[2] Greg J. Badros, Alan Borning, and Peter J. Stuckey. 2001. The Cassowary linear arithmetic constraint solving algorithm. *ACM Transactions on Computer-Human Interaction*, 8, 4, 267–306. DOI: 10.1145/504704.504705.

[3] Esma Nur Berdan. 2025. Choosing the right data format: JSON, XML, YAML, or CSV? Medium. Retrieved March 3, 2026 from https://medium.com/@esmanurberdan/choosing-the-right-data-format-json-xml-yaml-or-csv-b60113697bf3.

[4] Deyu Bo, Xiao Wang, Yang Liu, Yuan Fang, Yawen Li, and Chuan Shi. 2023. A survey on spectral graph neural networks. *arXiv*. DOI: 10.48550/arxiv.2302.05631.

[5] BobLd. 2024. Documentlayoutanalysis: resources and libraries for document layout analysis. GitHub repository. Retrieved March 3, 2026 from https://github.com/BobLd/DocumentLayoutAnalysis.

[6] Jiahui Cen, Yan Zhang, and Simon Fong. 2025. On continuous optimization for constraint satisfaction problems. *arXiv preprint arXiv:2510.04480*. Retrieved March 2, 2026 from https://arxiv.org/pdf/2510.04480.

[7] Google Labs. 2024. Pomelli: exploring semantic style in automated user interfaces. Google Labs. Retrieved May 11, 2026 from https://labs.google/pomelli/about.

[8] Xiaocen Guo, Sijia Fu, and Di Zhu. 2025. Aesthetic quality evaluation of packaging design with graph neural networks and composition features. *Scientific Reports*, 15, 1, 36046. DOI: 10.1038/s41598-025-20046-1.

[9] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang, Enhua Wu, and Qi Tian. 2022. Vision GNN: an image is worth graph of nodes. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*. Retrieved March 3, 2026 from https://arxiv.org/abs/2206.00272.

[10] ITU-T. 1996. Methods for subjective determination of transmission quality. ITU-T Recommendation P.800. International Telecommunication Union, Geneva, Switzerland. Retrieved October 12, 2026 from https://www.itu.int/rec/t-rec-p.800.

[11] Yue Jiang, Wolfgang Stuerzlinger, Matthias Zwicker, and Christof Lutteroth. 2020. ORCSolver: an efficient solver for adaptive GUI layout with OR-constraints. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '20)*. ACM, 1–14. DOI: 10.1145/3313831.3376610.

[12] Yue Jiang, Changkong Zhou, Vikas Garg, and Antti Oulasvirta. 2024. Graph4gui: graph neural networks for representing graphical user interfaces. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. Retrieved October 9, 2026 from https://arxiv.org/abs/2404.13521.

[13] Tanakorn Lertrusdachakul, Phayung Meesad, and Sunantha Kittirachakul. 2025. Enhancing clustering performance: a hybrid generalized K-means approach. *International Journal of Advanced Research (IJAR)*, 13, 1. Retrieved March 3, 2026 from https://www.journalijar.com/uploads/2025/04/6821b3f719224_IJAR-50827.pdf.

[14] David G. Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60, 2, 91–110. DOI: 10.1023/B:VISI.0000029664.99615.94.

[15] Benjamin Matz, Kristian Mitchell, and Neill D. F. Campbell. 2023. Visual dna: representing and comparing images using distributions of neuron activations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*

(CVPR). IEEE, 23514–23523. Retrieved March 2, 2026 from https://arxiv.org/pdf/2304.10036.

[16] Gheorghe Alin Sabau and Alin Claudiu Zamfiroiu. 2024. Real-time document collaboration—system architecture and design. *Applied Sciences*, 14, 18, 8356. DOI: 10.3390/app14188356.

[17] The Business Research Company. 2025. *AI-Powered Design Tools Market Report 2025*. Research and Markets. Retrieved March 2, 2026 from https://www.researchandmarkets.com/reports/5971142/ai-powered-design-tools-market-report.

[18] David Torpey and Richard Klein. 2024. Affine transformation estimation improves visual self-supervised learning. *arXiv*. DOI: 10.48550/arxiv.2402.09071.

[19] UiPath. 2024. Studio - state machines. UiPath Documentation. Retrieved March 2, 2026 from https://docs.uipath.com/studio/standalone/2024.10/user-guide/state-machines.

[20] Robert L. Williams II and Lawrence S. Robbins. 2001. Hard Constraints in Optimization. Tech. rep. NASA Technical Reports Server (NTRS). Retrieved March 3, 2026 from https://ntrs.nasa.gov/citations/20080023913.

[21] Tasmia Zerin, Moumita Asad, B. M. Mainul Hossain, and Kazi Sakib. 2025. Repairing responsive layout failures using retrieval augmented generation. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME '25)*. arXiv: 2511.00678. DOI: 10.48550/arXiv.2511.00678.

CHAPTER 2

RELATED LITERATURE AND STUDIES

This chapter is divided into three parts: the review of related literature and studies, the synthesis of the state-of-the-art, and the gap bridged by the study. It serves to establish a foundation for the research, deepen the researchers' understanding of the field, and situate the study within the context of existing literature.

Graph-Based Relational Logic and Perceptual Extraction

The evolution of automated design frameworks has progressed from basic image matching toward high-precision perceptual analysis. While Shaharom and Tahar [2023] established the foundational reliability of SIFT and SURF algorithms for maintaining structural integrity and asset preservation during image matching, Garcia and Chen [2023] enhanced the precision of these systems by shifting toward the CIE L*a*b* color space for superior perceptual extraction. This transition facilitated a more human-centric approach to identifying brand signatures by separating luminance from chromaticity, thereby effectively reducing visual noise.

Building upon these extraction techniques, Lertrusdachakul, Meesad, and Kittirachakul [2025] addressed the mathematical limitations of earlier clustering models by proposing a Hybrid Generalized K-Means approach to stabilize color centroid determination. Finally, Huang, Chen, and Liu [2025] demonstrated the current versatility of these frameworks by applying intelligent palette mapping to automate complex, high-fidelity design transitions. This progression ensures that modern systems can perform stylistic changes while strictly adhering to the structural ”DNA” of the original brand identity.

Graph Neural Networks (GNN) in Spatial Prediction

The transition toward automated design systems began with the foundational challenge of representing implicit spatial logic through graph-structured data. While Lee, Jiang, Essa, Le, Gong, Yang, and Yang [2020] introduced the Neural Design Network (NDN) to demonstrate how graph convolution kernels could mimic human spatial reasoning, Nauata, Chang, and Cheng [2020] established that relational generative adversarial networks (House-GAN) could encode complex spatial adjacency as bubble diagrams. These early works effectively shifted the field away from rigid, rule-based systems toward models capable of learning the fluid relationships between design elements.

Building on these relational foundations, Nauata, Hosseini, and Chang [2021] evolved this technology into House-GAN++, a refinement network that moved beyond simple adjacency to demonstrate how GNNs could iteratively transform relational graphs into structurally consistent architectural layouts. By 2024, the application of GNNs expanded into specialized modeling; Jiang, Zhou, Garg, and Oulasvirta [2024] introduced Graph4GUI, utilizing bipartite graphs to achieve high-precision UI element alignment. This was further validated by Purohit, Vyas, and Gupta [2024], who confirmed that GNN-based optimization achieves superior performance in constrained residential setups compared to traditional metaheuristic baselines.

Most recently, Kamboj, Kumar, and Deep [2025] culminated this research by demonstrating an end-to-end pipeline in Graphs to Blueprints, capable of converting abstract graph nodes into consistent floor plans at a massive scale. Graph-encoded relational constraints ensure architecturally coherent outputs at scale. This progression—from the initial mimicking of human reasoning to the achievement of high-accuracy UI prediction and large-scale automated modeling—represents the complete evolution of the spatial logic required for modern design orchestration, moving from abstract concepts to a high-precision, mathematically-driven reality.

The development of document image understanding reached a significant turning point with the introduction of LayoutLM by Xu, Li, Cui, Huang, Wei, and Zhou [2020]. Their research established the methodology of treating text blocks, layout positions, and visual embeddings as semantic nodes, defending the principle that raw page images can be decomposed into meaningful units like paragraphs, figures, and tables. This foundational approach allowed graph-based models to reason over both textual and visual structures simultaneously. By 2022, the field advanced as Han, Wang, Guo, Tang, Wu, and Tian [2022] generalized node extraction beyond documents in their work Vision GNN. By splitting raw images into non-overlapping patches that serve as graph nodes, they demonstrated that patch-based extraction is semantically richer than traditional CNN grids. This methodology enables Graph Neural Networks (GNNs) to capture irregular object boundaries and complex spatial relationships with higher accuracy. Building upon these foundations, Wang, Xu, Cui, and Wei [2023] extended node extraction by explicitly modeling text, image, and structural components as heterogeneous nodes within a graph. This progression illustrates a clear trajectory: from the initial establishment of semantic node extraction in 2020, through the extension to general vision tasks in 2022, to the recent culmination in hybrid systems. These modern advancements enable GNNs to reason over complex document and image structures with significantly greater semantic fidelity.

Before a Graph Neural Network can reason over spatial layouts, the raw visual data must first be transformed into discrete, mathematical nodes. Recent literature demonstrates that combining high-speed object detection architectures with Optical Character Recognition (OCR) establishes a robust pipeline for Semantic Layout Parsing. A recent study by dos Santos Júnior, Paixão, and Alvarez [2025], evaluating the comparative performance of YOLOv8 for the layout analysis of document images, concluded that the YOLOv8 architecture provides state-of-the-art precision and speed for extracting structural bounding boxes across complex, heterogeneous layouts. Building upon this validated extraction method, this study passed the generated bounding boxes to the Tesseract OCR engine to retrieve the

corresponding textual representations. This hybridized extraction methodology unflattened raster images into the spatial embeddings and labelled text nodes required for downstream GNN processing.

Constraint Satisfaction Problems (CSP) in Design

The journey toward mathematically maintaining structural integrity during automated layout transformations began with addressing the limitations of traditional linear solvers. Jiang, Stuerzlinger, Zwicker, and Lutteroth [2020] developed the ORCSolver, an efficient system designed specifically for adaptive GUI layouts containing non-convex OR-constraints. This breakthrough allowed design frameworks to handle complex scenarios—such as selecting between disparate navigation styles based on viewport space—moving beyond static grids to establish the groundwork for responsive environments that preserve semantic relationships across platforms.

As the demand for more fluid user experiences grew, research shifted toward achieving sub-millisecond response times. Li, Li, Fu, and Cai [2024] introduced SMT-Layout, a MaxSMT-based approach engineered to support real-time interaction in complex interfaces. By formulating layout rules as Satisfiability Modulo Theories (SMT) problems, the system gained the ability to dynamically resolve both soft and hard constraints. This allowed design elements to resize instantly during user interaction, successfully bridging the gap between theoretical constraint solving and practical, high-speed GUI behavior.

The most recent advancements have focused on the dual pillars of scalability and automation. To address the computational demands of modern interfaces, Zhang, Maqrot, Mouysset, and Bortolaso [2025] introduced a CSP-based scalable framework that utilizes decomposition techniques to manage hundreds of interdependent variables without lag. Simultaneously, the manual effort required to define these relationships has been reduced through the work of Penco, Pintar, Vranić, and Šoštarić [2025], who developed a Large

Language Model-driven framework capable of translating natural language requirements into formal constraint models. Together, these developments represent a complete progression from solving specific adaptive logic to enabling the scalable, automated generation of complex, real-time design models.

Stateful Refinement and Iterative Systems

The development of sophisticated orchestration frameworks begins with the foundational selection of data formats capable of representing complex hierarchical structures. Berdan [2025] established the JSON format as a critical standard for lightweight data exchange, identifying its nested structure as uniquely suited for representing the diverse properties of UI nodes. This structural backbone enables seamless communication between frontend interfaces and backend Graph Neural Network (engines), setting the stage for more complex state management. Once this communication channel is established, the focus shifts to maintaining consistency during automated adjustments. Prymushko, Puchko, Yaroshynskyi, Sinko, Kravtsov, and Artemchuk [2025] addressed this by implementing Conflict-Free Replicated Data Types (CRDTs), which provide a mathematical guarantee for efficient state synchronization in distributed environments. By utilizing CRDTs, a design system can resolve conflicting layout changes without requiring a central coordinator, ensuring the design state remains consistent across all active environments.

As synchronization logic became more robust, research moved toward optimizing the speed of these transitions. Gomes, Rodrigues, and Baquero [2025] introduced the ConflictSync approach, which utilizes a mechanism for bandwidth-efficient synchronization of divergent states. By focusing on transmitting delta updates rather than full state snapshots, this methodology ensures that complex design graphs can be updated in real-time even across low-bandwidth connections. This efficiency gain proves critical in collaborative environments where multiple agents concurrently modify shared design states.

This technical efficiency is a prerequisite for the final stage of orchestration: the iterative improvement of the system itself. To evaluate this capability, Cheng, Tang, Ma, Hackett, He, Su, Beschastnikh, Huang, Ma, and Xu [2025] introduced SysMoBench, a comprehensive benchmark for evaluating AI performance when formally modeling complex real-world systems. Their research emphasizes that refinement is a series of stateful transitions that gradually eliminate design ”smells” or constraint violations. By integrating these iterative heuristics with high-speed synchronization, modern frameworks can ensure that every refinement cycle brings the layout closer to the target brand identity while maintaining the structural integrity of the original design intent.

The evolution of bi-directional state machines in user interface research can be traced from early cyber-physical synchronization studies to subsequent applications in human-facing systems. Liang, McGee, Menassa, and Kamat [2020] established the principle that digital twins and physical robots must maintain continuous reciprocal updates, effectively modeling the interaction as a bi-directional finite state machine. This work highlighted the fundamental necessity of two-way state propagation to ensure total fidelity between simulated models and physical environments.

Building on this foundation, Goldenberg and Tractinsky [2021] extended the theoretical defense of bi-directionality into the domain of graphical user interfaces. Their study demonstrated that multilingual interfaces—particularly those requiring transitions between right-to-left (RTL) and left-to-right (LTR) orientations—inherently demand two-way directional state transitions. They argued that such interfaces should be formally understood as bi-directional state machines where layout and text orientation dynamically toggle in response to user context. Taken together, these studies illustrate a clear progression: while the foundational work in 2020 defended bi-directional synchronization in cyber-physical systems, subsequent research in 2021 applied the same theoretical framework to multilingual UI design. Subsequent work further extended this framework beyond UI design, exploring how bi-directional state machines could support adaptive accessibility systems.

This trajectory grounds bi-directional state machines as a unifying principle that ensures consistency across both machine-to-machine and human-centered interfaces.

Orchestration and Real-Time Processing

The successful orchestration of automated layout generation within a live, interactive canvas necessitates strict computational performance boundaries. Du, Turner, Dzitsiuk, and Kowdle [2020] established that complex geometric operations must be processed within strict millisecond thresholds to prevent user interface occlusion or visual freezing, a principle critical for the viability of algorithmic design frameworks in real-world applications. Furthermore, Hu, Wang, and Wu [2025] identified that as generative artificial intelligence paradigms scale to support highly complex spatial layout designs, the computational trade-off between generating optimal configurations and maintaining real-time responsiveness becomes a primary architectural constraint. Consequently, the system must be engineered to resolve mathematical graph dependencies at interactive rates to ensure the orchestration engine does not bottleneck the graphical interface.

To bridge the gap between static algorithmic prediction and dynamic manual refinement, the architecture implements a Human-in-the-Loop (HITL) orchestration paradigm. While traditional end-to-end models often execute procedural planning in a computational vacuum, He, Huang, and Wang [2022] mathematically proved that computational architectures can co-evolve spatial structures alongside human experts through their iPLAN model. By accommodating free-form interactions such as localized element nudging or resizing, the system treats manual adjustments as definitive, hardened constraints on the canvas. Instead of passively generating a new coordinate array that erases human input, the algorithm proposes procedural suggestions and calculates spatial harmonizations around the user's localized edits. This ensures the orchestration engine remains a collaborative instrument rather than an autonomous override mechanism.

By synthesizing these high-speed constraints and HITL principles, the orchestration layer operated as a highly synchronized, event-driven ecosystem. To prevent computational bottlenecks and ensure the graphical interface did not suffer from visual occlusion or frame-rate degradation during manual manipulation, the framework decoupled continuous drag states from the mathematical solvers. Every manual human refinement acted as an asynchronous, post-interaction event trigger. Specifically, the system hardened the spatial parameters and invoked the underlying constraint solver to calculate optimal coordinate placements only upon the user's release of the element (the on-drop or on-resize-complete event hook). This event-driven approach ensured that the algorithm respected the interactive human inputs without overloading the CPU during fluid drag states, maintaining both global layout harmony and strict latency thresholds [14, 6, 13].

Algorithmic Evaluation and Aesthetic Scoring

The evaluation of automated layout architectures necessitates a robust synthesis of high-speed search algorithms and mathematically grounded aesthetic evaluators. Chen, Wang, and Tan [2021] identified that navigating the nearly infinite permutation space of graphical element placements poses a significant computational bottleneck, as standard linear search models and evolutionary algorithms frequently suffer from latency and stagnation. To resolve these inefficiencies, Vinod and Murthy [2022] utilized nature-inspired metaheuristic algorithms to drastically reduce the time-space complexity required for solving engineering layout problems. Specifically, Got, Moussaoui, and Zouache [2021] deployed hybrid filter-wrapper mathematical models, such as the Whale Optimization Algorithm, to treat feature selection as a multi-objective task, yielding superior convergence rates. Furthermore, Qais, Mirjalili, and Hasanien [2022] introduced geometry-based frameworks like the Circle Search Algorithm, which uses circular motion to optimize search trajectories.

Following the generation of spatial coordinates, the framework applies quantitative aesthetic evaluation to score the visual quality of the layouts. Zheng, Yang, and Zeng [2025] employed combined CNN-GNN architectures to extract local and global structural features, resulting in an objective compositional scoring system that aligns with strict mathematical rules. Similarly, Guo, Fu, and Zhu [2025] demonstrated the use of Graph Neural Networks to quantify human-centric design rules—such as whitespace and visual density—into objective metrics. These intelligent design systems utilize AI-driven aesthetic scores to guide evolutionary strategies, successfully automating layouts that balance functionality with visual harmony [16].

These computational evaluations build upon foundational energy-based layout formulations established by O'Donovan, Agarwala, and Hertzmann [2014], who demonstrated that graphic layout quality could be mathematically modeled through competing objective functions penalizing whitespace imbalance, boundary collisions, and visual hierarchy degradation. By formulating layout synthesis as an optimization landscape balancing spatial emphasis and negative space, their work established the mathematical basis for evaluating compositional harmony. Expanding this formulation into deep generative models, Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] formalized a standardized four-class structural taxonomy for commercial graphic design layouts (comprising logo, text, underlay, and embellishment components) and validated that computational layout generation models must be benchmarked against both objective spatial metrics and structured multi-evaluator designer panels to accurately capture perceptual validity.

Beyond structural alignment and whitespace ratios, modern automated design frameworks must also account for visual interference between generated elements and underlying background content. To prevent the degradation of text legibility caused by visual noise, the layout optimization pipeline of this study integrated dynamic saliency evaluations. Advancing GAN-based domain adaptation for image-aware layout generation, Xu, Zhou, Ge, and Xu [2026] quantified layout occlusion through a spatial clash metric (R_{shm})

that computes the perceptual difference in the background's saliency map before and after the application of text masks. Following the same principle, this study measured the background saliency beneath each text element and penalized text placed over visually salient regions (the Legibility Penalty, Formula 3.15). Because the saliency map was computed on each candidate's own composed background, the generated layout preserved the primary focal regions of the background while maintaining structural legibility, without relying on static, offline feature extraction.

To manage the output of real-time generation cycles, the system must prevent the presentation of stochastic or biased candidates. Li, Chen, and Wang [2023] implemented fairness-aware recommendation ranking algorithms to mathematically balance and sort generated outputs based on rigorous merit. Ultimately, Shi, Shang, and Qi [2023] emphasized the integration of deep generative models to bridge the gap between high-fidelity coordinate predictions and human-centric aesthetic preferences, a goal that Patnaik, Jain, Krishnamurthy, and Sarkar [2025] formalized as Aesthetic-Aware Preference Alignment (AAPA). Such alignment provides the necessary heuristic filtering to validate optimal design structures before they are presented to the user.

Synthesis of the State-of-the-Art

Current state-of-the-art in automated layout generation and graphic design orchestration demonstrates a definitive paradigm shift from manual, coordinate-based drafting toward intelligent, algorithmically-driven frameworks [20, 34, 37]. Both the established literature and the proposed ThemeSync framework share a foundational reliance on computational algorithms to synthesize spatial arrangements. However, a critical synthesis of the reviewed literature reveals a persistent dichotomy between offline, generative AI models and real-time, interactive design systems. Deep generative models and graph-based frameworks excel at mathematically predicting spatial logic but predominantly operate as static,

offline batch processors [26, 27]. ThemeSync departs from this passive generation model by embedding advanced computational algorithms into a live, stateful orchestration loop, transforming the canvas from a one-time predictive output into a continuously reactive environment.

This divergence is further evident in the treatment of visual identity and structural logic. Existing literature heavily emphasizes perceptually-conscious clustering models and feature matching algorithms—such as hybrid K-Means, SIFT, and SURF—to isolate dominant hues and stylistic markers with high precision [7, 35, 22]. Concurrently, Graph Neural Networks (GNNs) represent the current apex of relational spatial modeling, successfully predicting bounding boxes in offline environments [18, 45]. Yet, current frameworks frequently treat these domains in isolation; perceptual extraction is applied primarily to static image analysis, while structural GNNs independently predict layouts. ThemeSync synthesizes these disparate technologies to construct a unified, hybrid ”Visual DNA.” Current automated design frameworks frequently suffer from typographic hallucinations, as relying purely on Optical Character Recognition (OCR) or feature matching to autonomously extract exact font families from flattened images is computationally error-prone. ThemeSync bridges this gap by decoupling structural extraction from typographic assignment. While the system integrates perceptual extraction directly with relational graph inference to autonomously map colors and stylistic identities onto a live structural graph, the exact typographic parameters—such as font families, specific font weights, and baseline geometric sizing—are deterministically defined by the user prior to orchestration. This ensures that institutional branding remains mathematically consistent during automated transformations, utilizing techniques akin to smart palette mapping [15] to move beyond flat visual adjustments and achieve deep structural awareness.

The most significant technological limitation in existing automated design tools is the prevalence of ”destructive updating,” where generative models force a complete layout recalculation that irreversibly overwrites manual user refinements. While recent computer

science literature validates the necessity of Constraint Satisfaction Problems (CSP) for scalable spatial issues [23, 30, 43] and state machines for decentralized synchronization [25, 31, 9], their combined application to protect localized creative intent is notably absent in the design domain. ThemeSync bridges this gap by combining the incremental Cassowary constraint-solving algorithm [1] with a Bi-directional State Machine utilizing JSON-based object serialization [2]. Unlike traditional AI generators that act as ”fire-and-forget” black boxes, ThemeSync treats user interactions as strict ”Hard Constraints.” The serialized state machine ensures that these human-in-the-loop refinements survive typography updates and every later re-solve, utilizing the CSP solver to propagate non-destructive recalculations of the surrounding layout and eliminating layout fragility.

Finally, while the literature demonstrates that geometry-based metaheuristics [10, 38, 33] and CNN-GNN architectures [11, 44] can mathematically optimize and score layouts based on strict composition rules, finding an optimal layout programmatically does not guarantee subjective user satisfaction. Contemporary generative models frequently force a single, mathematically prioritized output onto the designer. ThemeSync fundamentally contrasts with this approach by synthesizing high-speed state-space exploration with Aesthetic-Aware Preference Alignment (AAPA) [29]. Rather than automating the designer out of the workflow, the framework utilizes these heuristic-quality filters and fitness functions to evaluate and rank parallel layout candidates when a layout is initialized [24]. This interactive suggestion mechanism maintains the critical ”human-in-the-loop” paradigm, proving that high-speed algorithmic orchestration and stateful human creative control can be seamlessly integrated into a single design ecosystem.

Gap Bridged by the Study

Existing research in graphic design automation has largely advanced along distinct trajectories, with studies either focusing on automated layout generation or supporting manual

designer adjustments, but rarely integrating both within a unified framework. As shown in a comprehensive survey on deep generative models [36], deep generative models are currently the most popular way to create intelligent layouts. However, these systems work as static, offline pipelines that act like generative black boxes, producing outputs that are structurally correct but cannot be changed by humans during the process. Any changes made by a user are automatically undone when the process is run again. This rigidity is further compounded by the absence of stateful memory in existing design tools, wherein no current system retains a record of designer intent, causing global operations such as theme or template updates to consistently and irreversibly erase localized refinements. Furthermore, even state-of-the-art diffusion-based layout models reduce generation to a purely predictive task, prioritizing mathematical constraints over the subjective aesthetic preferences of designers and failing to align outputs with the contextual demands of professional design practice [42, 29]. Taken together, these issues reveal a fundamental and unresolved gap between the operational efficiency of algorithmic automation and its capacity to support meaningful human creative control.

This study fills these gaps by introducing ThemeSync, a stateful graphic design orchestration framework that combines automated layout intelligence with non-destructive human refinement. In response to the offline processing limits identified in prior work [36], ThemeSync runs multi-relational GNN inference once when a layout is opened and keeps the Cassowary constraint-solving algorithm within a live orchestration loop, allowing structural changes to occur in real time while keeping relational dependencies intact. To resolve the destructive updating problem, the framework employs JSON-based state serialization and state machine logic to encode user interactions as hardened constraints, ensuring localized designer refinements persist through every later re-solve. Furthermore, contemporary deep generative layout models frequently suffer from semantic ambiguity when merging static structural templates with variable data. ThemeSync bridges this gap by decoupling the aesthetic extraction pipeline from a dynamic payload injection system.

By utilizing Dynamic Node Instantiation and Implicit Geometric Hierarchy, the framework ensures that user-defined administrative variables—such as specific names or titles—are placed in a defined reading order and assigned role-specific baseline dimensions, allowing dynamic content to be logically orchestrated alongside the master theme. Building on aesthetic alignment principles established in recent literature [29], ThemeSync integrates Aesthetic-Aware Preference Alignment (AAPA) into an interactive recommendation dashboard, replacing single programmatic outputs with a set of fitness-ranked layout candidates. ThemeSync changes the role of AI in graphic design from an independent generator to a human-in-the-loop collaborative tool that respects both structural logic and creative intent.

Notes

[1] Greg J. Badros, Alan Borning, and Peter J. Stuckey. 2001. The Cassowary linear arithmetic constraint solving algorithm. *ACM Transactions on Computer-Human Interaction*, 8, 4, 267–306. DOI: 10.1145/504704.504705.

[2] Esma Nur Berdan. 2025. Choosing the right data format: JSON, XML, YAML, or CSV? Medium. Retrieved March 3, 2026 from https://medium.com/@esmanurberdan/choosing-the-right-data-format-json-xml-yaml-or-csv-b60113697bf3.

[3] Jie Chen, Xiulin Wang, and Ying Tan. 2021. Differential evolution with adaptive mutation strategy based on fitness landscape analysis. *Information Sciences*, 549, 142–163. DOI: 10.1016/j.ins.2020.11.023.

[4] Qian Cheng, Ruize Tang, Emilie Ma, Finn Hackett, Peiyang He, Yiming Su, Ivan Beschastnikh, Yu Huang, Xiaoxing Ma, and Tianyin Xu. 2025. SysMoBench: evaluating AI on formally modeling complex real-world systems. Retrieved March 3, 2026 from https://arxiv.org/abs/2509.23130.

[5] Eder Silva dos Santos Júnior, Thuanne Paixão, and Ana Beatriz Alvarez. 2025. Comparative performance of yolov8, yolov9, yolov10, and yolov11 for layout analysis of historical documents images. *Applied Sciences*, 15, 6, 3164. DOI: 10.3390/app15063164.

[6] Ruofei Du, Eric Turner, Maxim Dzitsiuk, and Adarsh Kowdle. 2020. DepthLab: real-time 3D interaction with depth maps for mobile augmented reality. In *Proceedings of the UIST '20*. ACM, 829–843. DOI: 10.1145/3379337.3415881.

[7] Miguel Garcia and Jia-Ray Chen. 2023. New approach to dominant color extraction in images with wide range of hues using CIE lab space. *Technologies*, 11, 6, 230. DOI: 10.3390/technologies11060230.

[8] Y. Goldenberg and N. Tractinsky. 2021. Towards the right direction in bidirectional user interfaces. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI 2021)*. ACM. Retrieved March 2, 2026 from https://dl.acm.org/doi/10.1145/3411764.3445461.

[9] Pedro Silva Gomes, Miguel Boaventura Rodrigues, and Carlos Baquero. 2025. ConflictSync: bandwidth efficient synchronization of divergent state. *arXiv*. Retrieved March 4, 2026 from https://arxiv.org/abs/2505.01144.

[10] Abdellah Got, Abdelouahab Moussaoui, and Djaafar Zouache. 2021. Hybrid filter-wrapper feature selection using whale optimization algorithm: a multi-objective approach. *Expert Systems with Applications*, 183, 115312. DOI: 10.1016/j.eswa.2021.115312.

[11] Xiaocen Guo, Sijia Fu, and Di Zhu. 2025. Aesthetic quality evaluation of packaging design with graph neural networks and composition features. *Scientific Reports*, 15, 1, 36046. DOI: 10.1038/s41598-025-20046-1.

[12] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang, Enhua Wu, and Qi Tian. 2022. Vision GNN: an image is worth graph of nodes. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*. Retrieved March 3, 2026 from https://arxiv.org/abs/2206.00272.

[13] Feixiang He, Yanlong Huang, and He Wang. 2022. Iplan: interactive and procedural layout planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7783–7792. DOI: 10.1109/CVPR52688.2022.00764.

[14] Fuwen Hu, Chun Wang, and Xuefei Wu. 2025. Generative artificial intelligence-enabled facility layout design paradigm. *Applied Sciences*, 15, 10, 5697. DOI: 10.3390/app15105697.

[15] Haoran Huang, Yong Chen, and Zhaolin Liu. 2025. Recoloring cartoon images based on palette mapping using K-means. *Engineering Proceedings*, 10, 7, 82. DOI: 10.3390/engproc2025107082.

[16] Zhe Ji and Yan Yu. 2025. Intelligent interior design systems: optimizing layouts and aesthetics using ai and user data. *IEEE Access*, 13, 135127–135137. DOI: 10.1109/ACCESS.2025.3591135.

[17] Yue Jiang, Wolfgang Stuerzlinger, Matthias Zwicker, and Christof Lutteroth. 2020. ORCSolver: an efficient solver for adaptive GUI layout with OR-constraints. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '20)*. ACM, 1–14. DOI: 10.1145/3313831.3376610.

[18] Yue Jiang, Changkong Zhou, Vikas Garg, and Antti Oulasvirta. 2024. Graph4gui: graph neural networks for representing graphical user interfaces. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. Retrieved October 9, 2026 from https://arxiv.org/abs/2404.13521.

[19] Arshita Kamboj, Rahul Kumar, and Aman Deep. 2025. Graphs to blueprints: GNN-powered floor plan modeling. *International Advanced Research Journal in Science, Engineering and Technology (IARJSET)*, 12, 2. Retrieved March 3, 2026 from https://iarjset.com/wp-content/uploads/2025/03/IARJSET.2025.12227.pdf.

[20] Min-Ji Kim, Hyun-Jung Lee, and Sang-Hee Park. 2025. From fragment to one piece: a review on AI-driven graphic design. *Journal of Imaging*, 11, 9. DOI: 10.3390/jimaging11090289.

[21] Hsin-Ying Lee, Lu Jiang, Irfan Essa, Phuong B. Le, Haifeng Gong, Ming-Hsuan Yang, and Weilong Yang. 2020. Neural design network: graphic layout generation with constraints. In *Proceedings of the European Conference on Computer Vision (ECCV '20)*. Retrieved October 9, 2026 from https://arxiv.org/abs/1912.09421.

[22] Tanakorn Lertrusdachakul, Phayung Meesad, and Sunantha Kittirachakul. 2025. Enhancing clustering performance: a hybrid generalized K-means approach. *International Journal of Advanced Research (IJAR)*, 13, 1. Retrieved March 3, 2026 from https://www.journalijar.com/uploads/2025/04/6821b3f719224_IJAR-50827.pdf.

[23] Bohan Li, Dawei Li, Ming Fu, and Shaowei Cai. 2024. SMT-layout: a MaxSMT-based approach supporting real-time interaction of real-world GUI layout. *arXiv preprint arXiv:2411.12271*. Retrieved March 3, 2026 from https://arxiv.org/abs/2411.12271.

[24] Yunqi Li, Li Chen, and Jun Wang. 2023. Fairness in recommendation: foundations, methods, and applications. *ACM Trans. Intell. Syst. Technol.*, 14, 5, 79. DOI: 10.1145/3610302.

[25] Cheng Liang, Wesley McGee, Carol Menassa, and Vineet Kamat. 2020. Bi-directional communication bridge for state synchronization between digital twin simulations and physical construction robots. In *Proceedings of the International Symposium on Automation and Robotics in Construction (ISARC 2020)*. DOI: 10.22260/ISARC2020/0205.

[26] Nelson Nauata, Kai-Hung Chang, and Cheng-Yi Cheng. 2020. House-GAN: relational generative adversarial networks for graph-constrained house layout generation. In *Proceedings of the European Conference on Computer Vision (ECCV '20)*. Retrieved March 3, 2026 from https://arxiv.org/abs/2003.06988.

[27] Nelson Nauata, Sepidehsadat Hosseini, and Kai-Hung Chang. 2021. House-GAN++: generative adversarial layout refinement network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '21)*. Retrieved March 3, 2026 from https://arxiv.org/abs/2103.02574.

[28] Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. 2014. Learning layouts for single-page graphic designs. *IEEE Transactions on Visualization and Computer Graphics*, 20, 8, 1200–1213. DOI: 10.1109/TVCG.2014.48.

[29] Sohan Patnaik, Rishabh Jain, Balaji Krishnamurthy, and Mausoom Sarkar. 2025. Aesthetiq: enhancing graphic layout design via aesthetic-aware preference alignment of multi-modal large language models. In *Proceedings of the CVPR '25*. IEEE, 23701–23711. DOI: 10.1109/CVPR52734.2025.02207.

[30] Roberto Penco, Damir Pintar, Mihaela Vranić, and Marko Šoštarić. 2025. Large language model-driven framework for automated constraint model generation in configuration problems. *Applied Sciences*, 15, 12, 6518. DOI: 10.3390/app15126518.

[31] Arsentii Prymushko, Ivan Puchko, Mykola Yaroshynskyi, Dmytro Sinko, Hryhoriy Kravtsov, and Volodymyr Artemchuk. 2025. Efficient state synchronization in distributed electrical grid systems using conflict-free replicated data types. *IoT*, 6, 1, 6. DOI: 10.3390/iot6010006.

[32] Jignesh Purohit, Manju Vyas, and Sandeep Gupta. 2024. An application of graph neural network model design for residential building plan layout. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15, 11. DOI: 10.14569/IJACSA.2024.0151130.

[33] Mohammed H. Qais, Seyedali Mirjalili, and Hany M. Hasanien. 2022. Circle search algorithm: a geometry-based metaheuristic optimization algorithm. *Mathematics*, 10, 10, 1626. DOI: 10.3390/math10101626.

[34] Adrian G. Sanchez, Juan M. Rivera, and Felipe L. Torres. 2025. Artificial intelligence in generative design: a structured review of trends and opportunities in techniques and applications. *Designs*, 9, 4. DOI: 10.3390/designs9040079.

[35] M. Shaharom and K. Tahar. 2023. Multispectral image matching using SIFT and SURF algorithm: a review. *International Journal of Geoinformatics*, 19, 1, 13–21. Retrieved March 3, 2026 from https://ijg.e-geoinfo.com/index.php/journal/article/view/2495/1329.

[36] Yong Shi, Mengyuan Shang, and Zhiquan Qi. 2023. Intelligent layout generation based on deep generative models: a comprehensive survey. *Information Fusion*, 100, 101940. DOI: 10.1016/j.inffus.2023.101940.

[37] Marco van Hurne. 2026. From Suitability to Blueprint: A Unified Framework for Agentic AI Process Automation in Enterprise Environments. Tech. rep. Eigenvector Research. DOI: 10.13140/RG.2.2.29631.16800.

[38] V. C. Vinod and A. H. S. Murthy. 2022. Nature inspired meta heuristic algorithms for optimization problems. *Computing*, 104, 2, 251–269. DOI: 10.1007/s00607-021-00955-5.

[39] Zhiyuan Wang, Yiheng Xu, Lei Cui, and Furu Wei. 2023. A graphical approach to document layout analysis. In *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR 2023)*. Retrieved March 3, 2026 from https://ar5iv.labs.arxiv.org/html/2308.02051.

[40] Chenchen Xu, Min Zhou, Tiezheng Ge, and Weiwei Xu. 2026. Gan-based domain adaptation for image-aware layout generation in advertising poster design. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 48, 1, 184–201. DOI: 10.1109/TPAMI.2025.3602846.

[41] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. LayoutLM: pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)*. ACM, 1192–1200. Retrieved March 3, 2026 from https://arxiv.org/abs/1912.13318.

[42] Junyi Zhang, Jiaqi Guo, Shizhao Sun, Jian-Guang Lou, and Dongmei Zhang. 2023. Layoutdiffusion: improving graphic layout generation by discrete diffusion probabilistic models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 7226–7236. Retrieved October 12, 2026 from https://arxiv.org/abs/2303.11589.

[43] Liwen Zhang, Sara Maqrot, Florent Mouysset, and Christophe Bortolaso. 2025. A constraint satisfaction problems based scalable framework to address large-scale realistic problems. In *Proceedings of the ICORES 2025*. SciTePress, 45–56. Retrieved March 3, 2026 from https://www.scitepress.org/Papers/2025/131096/131096.pdf.

[44] Zixuan Zheng, Ding Yang, and Li Zeng. 2025. A deep learning framework for objective aesthetic evaluation of indoor landscapes using CNN-GNN model. *Scientific Reports*, 15. DOI: 10.1038/s41598-025-24548-w.

[45] Jie Zhou, Ganqu Cui, and Shengding Hu. 2020. Graph neural networks: a review of methods and applications. *AI Open*, 1, 57–81. DOI: 10.1016/j.aiopen.2021.01.001.

[46] Min Zhou, Chenchen Xu, Ye Ma, Tiezheng Ge, Yuning Jiang, and Weiwei Xu. 2022. Composition-aware graphic layout gan for visual-textual presentation designs. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*. International Joint Conferences on Artificial Intelligence Organization, 4995–5001. DOI: 10.24963/ijcai.2022/692.

CHAPTER 3

METHODOLOGY

This chapter discussed the research methods, procedures, and techniques that were used to systematically identify, select, process, and analyze information to address the specific problems posed in this study. It explained theorems, algorithms, and mathematical models employed, the materials and statistical tools that were used, the theoretical/conceptual framework, and the methods for analyzing data to ensure accurate interpretation of results.

Research Design

This study employed a Constructive Research Design, a methodological framework specifically utilized in Computer Science and software engineering to solve domain-specific problems through the creation of novel computational artifacts, algorithms, or mathematical architectures. Unlike descriptive or exploratory research that primarily observed existing phenomena, the constructive approach focused on the systematic development and rigorous validation of a proposed solution. This design dictated that the research had to yield a tangible, functional construct—in this case, an algorithmic framework—that addressed a defined technical gap, followed by an objective, mathematically grounded evaluation to prove the artifact's practical utility and theoretical soundness within the discipline.

In application to this study, observing or evaluating current design systems was insufficient to solve the ”Generative Gap” and ”Layout Fragility” identified in contemporary graphic design automation. Because existing industrial tools relied on static templates that caused destructive state management, solving this problem required the active construction of the ThemeSync orchestration architecture. By utilizing a constructive methodology, the researchers systematically engineered the tri-layer pipeline, integrating Graph Neural

Networks (GNN) for spatial mapping with a Cassowary linear arithmetic solver and a Bi-directional State Machine for non-destructive, real-time recalculations. Furthermore, to bridge the gap between theoretical computation and practical application, the design integrated a mixed-methods evaluation paradigm. The framework's validity was established by first mathematically proving the artifact's structural integrity against a Heuristic Fitness Function, followed by a quantitative human-expert validation using a Mean Opinion Score (MOS) to assess its professional aesthetic quality.

Theorems, Algorithms, and Mathematical Framework

This section delineated the foundational theorems, computational algorithms, and mathematical models that drove the ThemeSync orchestration framework. Specifically, it formalized the underlying architecture of the Graph Neural Networks (GNN) utilized for probabilistically mapping spatial dependencies. Furthermore, this section explicitly defined the system of linear inequalities and objective functions governing the Cassowary constraint-solving algorithm, establishing the mathematical boundaries required to dynamically calculate optimal layout coordinates while strictly preserving user-defined geometric parameters.

Perceptually-Consistent Color Quantization (K-Means Clustering)

ThemeSync used K-Means Clustering in the CIE color space L*a*b, which is a perceptually uniform color space, unlike the non-linear RGB color space which is device-dependent, with the Euclidean distance between two points in the space directly proportional to the perceived difference between them by human vision. The algorithm divided the number of pixels in a picture, n, into k color clusters by minimizing the **Within-Cluster Sum of Squares (WCSS):** as shown in the formula below:

$$J = \sum_{i=1}^{k} \sum_{x_p \epsilon S_i} ||x_p - \mu_i||^2 \tag{3.1}$$

The extraction process defines x as the $L^*a^*b^*$ feature vector of an individual pixel, which is then grouped into clusters where i represents the mean of pixels within a specific cluster S_i [15]. These clusters ultimately determine k, the target color centroids representing the primary, secondary, and accent colors. By grounding the analysis in the $L^*a^*b^*$ color space, the system ensures that the resulting palette remains consistent across diverse display technologies; this is achieved because the Euclidean distance calculated between clusters through Formula 3.1 correlates directly with the visual difference perceived by the human eye [5].

Stylistic Marker Identification (SIFT)

To identify the master's distinctive visual features and the visually busy regions of its background, the system employed the Scale-Invariant Feature Transform (SIFT). SIFT extracts local keypoints that are resistant to scaling, rotation, and affine transformation [27]. Mathematically, these keypoints are identified through a Difference of Gaussians (DoG) function as shown in Formula 3.2:

$$\mathbf{D}(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * \mathbf{I}(x, y) \tag{3.2}$$

Because these keypoints are invariant to scale, their density measured how much distinctive visual detail a region contained, independently of the size of the Master Design [27]; the logos themselves were located by the layout detector. The framework used SIFT to construct a mathematical Saliency Map of the inpainted background layer. In this computational pipeline, the inpainted background referred to a text-free base layer mathematically rebuilt from the borders of the master poster. In addition to this base layer, the system isolated specific stylistic markers—namely, the primary title artwork and decorative motifs (which were detected via YOLO and ranked by SIFT keypoint density)—to ensure these

distinct visual assets were preserved and transferred to the target format. By aggregating the high-frequency keypoints detected via the DoG function—such as sharp gradients, dense textures, or disruptive visual noise—the system generates a spatial heat map. The orchestration engine subsequently utilizes this Saliency Map to calculate a Legibility Penalty, ensuring that dynamically injected text nodes are mathematically restricted from occupying visually noisy coordinate spaces.

Linear Arithmetic Constraint Solving (The Cassowary Algorithm)

The Cassowary Algorithm serves as the core engine for real-time layout optimization. It is optimized to solve systems of linear equalities and inequalities efficiently [1]. Every design element is represented by a set of variables: $x_i\,(left)$, $y_i\,(top)$, $w_i\,(width)$, $h_i\,(height)$. Relational logic (e.g., margins and padding) is expressed through linear equations defined in Formula 3.3:

$$\begin{aligned} x_{\text{logo}} &= x_{\text{boundary}} + 20 \\ x_{\text{right}} &= x_{\text{left}} + \text{width} \end{aligned} \tag{3.3}$$

The solver used a dual-simplex method to satisfy the relationships established in Formula 3.3. In this implementation, the whole layout was re-solved on every terminal move, resize, or lock, and each solve completed within milliseconds, so the layout re-calculated immediately whenever a variable—such as an element's position or the viewport width—was adjusted [16].

Stateful Human-in-the-Loop Refinement

The framework differentiates between two types of constraints based on mathematical priority to support real-time interaction [16]:

1. **Hard Constraints:** Manual user interactions, such as "nudging" a text box or locking a logo's position, are assigned infinite weight. These are strictly prohibited from

being violated by the solver and are mathematically represented as a Hard Equality Constraint in Formula 3.4:

$$C_{\text{hard}} : f(x) = b \tag{3.4}$$

2. **Soft Constraints:** Standard design rules (e.g., default margins) or suggested alignments are assigned finite weights (w_i). The solver attempts to satisfy these by minimizing the total error through a Weighted Objective Function defined in Formula 3.5:

$$\textbf{minimize} \sum w_i d_i \tag{3.5}$$

Where d_i represents the "slack" or distance from the target suggested state [24].

Graph Neural Networks (GNN)

The architecture of a Graph Neural Network (GNN) was designed for processing relational data by maintaining and updating node states through a graph structure [31]. In the ThemeSync framework, the design canvas was modeled as a graph structure. Each design element—such as a text block, logo, or image—functioned as a node, while spatial and semantic relationships—such as alignment or proximity—were represented as edges. Through a message-passing mechanism, nodes exchanged information with their neighbors, allowing the system to capture dependencies among elements. This process enabled the model to infer how design components interact and depend on one another, supporting relational logic inference for layout optimization.

Initial Feature Encoding

Every design element started with a set of basic information called a feature vector [33]. This initial description recorded the element's spatial and visual properties, serving as the starting point for relational analysis as established in Formula 3.6:

$$h_i^{(0)} = [x_i,\ y_i, w_i, h_i, c_i, s_i] \tag{3.6}$$

In this formulation provided by the formula above, (x_i, y_i) represents the specific position of the element on the canvas, while (w_i, h_i) denotes its corresponding width and height dimensions. The term c_i refers to the CIE $L^*a^*b^*$ color embedding extracted via K-Means clustering, and s_i to the SIFT saliency weight. In the implemented model, both were taken from the layout as a whole, as its primary color centroid and its overall saliency weight, and were therefore shared by all the nodes of a layout. Because the baseline training dataset (CGL-Dataset) categorizes text under a generic supercategory rather than explicit semantic labels, the framework established visual priority through Implicit Geometric Hierarchy. During Dynamic Node Instantiation, the system algorithmically assigned Baseline Geometric Tensors (predefined bounding box dimensions, $w_i \times h_i$) to external administrative variables before they were encoded into the graph, and the Spatial Hierarchy Index fixed their reading order. Visual priority was therefore carried by each element's role-specific dimensions and its position in the reading order, both of which entered the GNN through Formula 3.6, rather than by explicit semantic tagging. By organizing these variables into the structure of Formula 3.6, the system created a standardized input for the subsequent Graph Neural Network layers.

Relational Inference via Message Passing

The GNN processed these features through a layer-wise exchange [2]. At each layer k, every element computed messages from its neighbors to update its own representation as defined in Formula 3.7:

$$\begin{aligned} m_{ij}^{(k)} &= \phi(h_i^{(k)},\ h_j^{(k)},\ e_{ij}) \\ h_i^{(k+1)} &= \psi(h_i^{(k)}, AGGm_{ij}^{(k)} :\ j\ \epsilon\ N(i)) \end{aligned} \tag{3.7}$$

In this architecture described by Formula 3.7, ϕ represents the message function, implemented as a multilayer perceptron (MLP) designed to compute pairwise relational features

between adjacent nodes. The AGG term denotes a permutation-invariant aggregation operator, such as mean or sum pooling, while ψ serves as the node update function, utilized as a Gated Recurrent Unit (GRU) to update the hidden state of each node.

The connections e_{ij} utilized within the message function of Formula 3.7 carried edge features describing spatial logic, which are formally structured in Formula 3.8:

$$e_{ij} = [\delta x_{ij}, \delta y_{ij}, IoU_{ij}, r_{ij}^{align}] \tag{3.8}$$

According to Formula 3.8, these features include the horizontal and vertical offsets, Intersection over Union (IoU), and an alignment score, where 0 means no alignment and 1 means perfect alignment.

To optimize relational spatial logic, the Graph Neural Network (GNN) was trained to predict layout coordinates by utilizing the master design's structure as its initial input. During initialization, target elements that shared a semantic counterpart on the master layout (e.g., the logo row, header, and event details) were anchored at their counterpart's original positions, while unmapped elements were positioned in reading order between them; in the absence of a master layout, elements defaulted to a centered, naive stack. The model was trained on CGL-Dataset layouts from both initialization states, deliberately distorting the structure starts during training to simulate real cross-format mapping transformations. The final GNN output was registered back to the master's anchors, ensuring that the mapped elements retained their original placements while the network placed the unmapped elements between them. On a held-out validation split of 2,900 CGL-Dataset layouts (a seeded 5% of its training split, since the dataset's test split carries no annotations), the GNN reduced the position error of the structure start from 0.050 to 0.036, expressed as fractions of the canvas. When the master's exact anchors were given, the registered output placed the unmapped elements with a vertical-center error of 0.075, compared with 0.081 for linear interpolation between the anchors.

State Synchronization — Finite State Machine and JSON Schema

The State Synchronization module recorded every definitive change the designer made during a session. Each interaction—whether a finalized position nudge, a completed resize, or an element lock—was saved as an immutable entry in a JSON-based state chain [25]. No prior state was ever overwritten, so the designer's complete editing history was preserved and any earlier version of the canvas remained restorable. To ensure optimal computational latency, the state machine operated strictly as an Event-Driven Post-Interaction Solver. The system deliberately ignored continuous manipulation data (e.g., on-drag coordinate streams) to prevent overloading the solver pipeline. Instead, a state transition was initiated only upon a terminal user action. When a discrete event was performed at time t, the system moved from the current canvas state S_t to a new state S_{t+1} based on a deterministic transition function defined in Formula 3.9 [13].

$$S_{t+1} = \delta(S_t, e_t) \tag{3.9}$$

In the model represented by Formula 3.9, S_t represents the current canvas state at time t, while e_t denotes the specific, terminal interaction event performed by the designer (which includes on-drop coordinate shifts, on-resize-complete dimensional changes, lock toggles, and direct text edits). The transition function is defined by δ, which processes the current state and the discrete event to trigger the Cassowary recalculation for a move, resize, or lock and produce the subsequent stabilized state of the canvas; a text edit was re-validated against the content schema and re-drawn in its box.

Furthermore, to preserve the structural integrity of the append-only JSON state chain, undo and redo actions executed on the canvas did not rewrite or erase historical layout data; instead, they restored an earlier configuration by appending it as a newly generated state.

After each transition, every affected element was assigned a constraint type [19]. The classification logic for these elements is formally presented in Formula 3.10:

$$c_i = \text{Hard}, \quad \text{if } e_i \in U_{manual}$$
$$c_i = \text{Soft}, \quad \text{if } e_i \in A_{algorithmic} \tag{3.10}$$

As specified in Formula 3.10, elements manually adjusted by the designer (U_{manual}) were labeled as Hard constraints, meaning the system did not change their position or dimensions. Conversely, elements placed or adjusted by the algorithm ($A_{algorithmic}$) were labeled as Soft constraints, which the system was permitted to modify during layout optimization.

Validation of Hybrid Visual DNA Correctness

To guarantee that the orchestration engine executed with structural and stylistic precision, the framework implemented a dual-validation protocol to mathematically and programmatically ensure the correctness of the Hybrid Visual DNA prior to layout synthesis. Because the Visual DNA is composed of both extracted and injected variables, the validation was bifurcated into Algorithmic Correctness and Administrative Correctness.

The correctness of the autonomously extracted color features was mathematically validated by optimizing the Within-Cluster Sum of Squares (WCSS) during the K-Means execution, as established in Formula 3.1. To prevent the extraction of inaccurate or perceptual outlier colors, the algorithm iteratively computed the WCSS across multiple centroid configurations ($k = 2$ to 6, each the best of five seeded initializations, so that a master always yielded the same palette). The system programmatically identified the optimal k-value by detecting the inflection point (Elbow Method) in the WCSS decay curve. This mathematical convergence served as the validation metric, ensuring that the extracted color centroids represented the perceptual variance of the master design template.

To prevent the typographic hallucinations inherent in traditional Optical Character Recognition (OCR) systems, the correctness of the user-defined typographic parameters (font family, font weight, and baseline geometric sizing) was strictly enforced through pre-

orchestration JSON Schema Validation. Prior to initializing the Graph Neural Network (GNN) or the Cassowary solver, the framework parsed the ingested Dynamic Content Payload against a rigid, predefined data schema. The validation algorithm verified that all inputs strictly conformed to programmatic constraints formalized via JSON Schema (Draft 2020-12). Specifically, typographic rules dictated that font weights belonged to the integer set $\{100, 200, \ldots, 900\}$, and baseline font sizes were constrained between a minimum of 8 pixels and a maximum of 15% of the viewport's shorter side. Furthermore, content payload rules required that every requested schema variable was present and formatted strictly as a string that did not exceed its predefined `max_characters` limit. If a user payload violated these structural boundaries, the system programmatically rejected the initialization, thereby guaranteeing deterministic typographic correctness and preventing downstream layout failure.

Quantitative Layout Evaluation (Heuristic Fitness Function)

To programmatically evaluate the structural and aesthetic validity of the generated coordinate layouts without relying on subjective human surveying, the framework implements a mathematically rigorous Heuristic Fitness Function. This objective function is formulated as a minimization problem, calculating an aggregate penalty score based on four distinct spatial constraints. The overarching fitness function is defined in Formula 3.11:

$$F_{layout} = \alpha P_{collision} + \beta P_{alignment} + \gamma P_{whitespace} + \delta P_{legibility} \quad (3.11)$$

In Formula 3.11, the coefficients $(\alpha, \beta, \gamma, \delta)$ represent static scalar weights empirically calibrated during Phase 3 testing to prioritize critical layout rules (e.g., α is assigned the highest weight to strictly penalize element overlap). The individual penalty variables are mathematically defined as follows:

1. **Collision Penalty ($P_{collision}$):** This metric calculates the absolute area of spatial overlap between all independently generated bounding boxes (B). It ensures elements do not visually occlude one another. The total overlap is normalized by the canvas area, and overlaps between two decorative motifs are not counted, as defined in Formula 3.12:

$$P_{collision} = \frac{1}{Area_{canvas}} \sum_{i=1}^{N} \sum_{j=i+1}^{N} Area(B_i \cap B_j) \tag{3.12}$$

2. **Alignment Penalty ($P_{alignment}$):** This metric calculates the spatial variance of generated nodes against a dynamically generated structural grid (G). It computes the minimum horizontal distance from a node's left edge, center, or right edge (the set X_i) to the nearest vertical gridline, normalized by the canvas width W_{canvas} and averaged over the N nodes; signature blocks and decorative motifs, which are placed off-grid by design, are excluded. It is defined in Formula 3.13:

$$P_{alignment} = \frac{1}{N\, W_{canvas}} \sum_{i=1}^{N} \min_{x \in X_i,\, g \in G} |x - g| \tag{3.13}$$

The structural grid G was dynamically constructed from the layout candidate's specific content area—spanning its margins, quarters, and center—and an element was evaluated as aligned when its left edge, center, or right edge lay on a gridline.

3. **Whitespace Penalty ($P_{whitespace}$):** This metric evaluates the mathematical ratio of negative space to positive design elements, measuring its deviation from a target ratio (R_{target}), which was fixed at 0.65 for every master layout. It is defined in Formula 3.14:

$$P_{whitespace} = \left| \frac{Area_{canvas} - \sum_{i=1}^{N} Area(B_i)}{Area_{canvas}} - R_{target} \right| \tag{3.14}$$

For the whitespace ratio calculation, the reference area was strictly defined as the candidate's designated content region (e.g., the active text column within an art-column archetype) rather than the global page canvas. This localized area calculation ensured that asymmetric or split layouts were accurately evaluated for negative space without being inadvertently penalized for off-center spatial compositions.

4. **Legibility Penalty ($P_{legibility}$):** This metric prevents text degradation by cross-referencing the predicted bounding boxes of text nodes (B_t) against the background Saliency Map (S) generated via SIFT. It calculates the mean saliency $S(p)$ of the pixels p underlying each text box, averaged over the set of text nodes T so that the term stays on the same scale as the other penalties, as defined in Formula 3.15:

$$P_{legibility} = \frac{1}{|T|} \sum_{t \in T} \frac{1}{|B_t|} \sum_{p \in B_t} S(p) \tag{3.15}$$

By calculating these interconnected formulas, the orchestration engine scored every candidate configuration and ranked the candidates by their F_{layout} value; Phase 3 describes how the final output was selected among them.

Materials and Evaluation Methods

This section outlined the comprehensive methodology utilized to construct, validate, and empirically evaluate the ThemeSync orchestration framework. It detailed the specific computational hardware, software environments, and foundational datasets required as the primary research instruments. Furthermore, this section defined the systematic procedures for both the automated algorithmic testing and the expert aesthetic validation, alongside the purposive sampling of the evaluation posters. Finally, it established the precise statistical tools and overarching evaluation methods necessary to quantitatively measure algorithmic latency, constraint satisfaction rates, and expert-rated aesthetic quality, ensuring a rigorous and multi-dimensional assessment of the proposed architecture.

Research Instruments and Data Gathering

The primary instruments for this algorithmic study comprised specialized computational datasets and structured vector templates. These instruments provided the foundational data required to train the relational logic models and empirically test the real-time constraint solvers.

Training Data and Feature Mapping

To process visual layouts computationally and establish implicit spatial knowledge, the framework utilized as its primary offline training corpus the CGL-Dataset V2, which Li, Liu, Feng, Zhu, Li, Zhang, Lv, Zhu, Shen, Lin, and Shao [2023] introduced as an extension of the CGL-Dataset established by Zhou, Xu, Ma, Ge, Jiang, and Xu [2022]. This dataset comprises 60,548 training and 1,035 test graphic design posters annotated across four structural supercategories: logo, text, underlay, and embellishment. This formalized corpus provided the requisite structural bounding boxes and spatial adjacency patterns necessary to train the Graph Neural Network (GNN) for relational layout inference.

Because the raw CGL-Dataset was structured in a tabular delivery format containing comprehensive administrative metadata (e.g., image_id, file_name, and nested annotations), the framework enforced a deterministic preprocessing pipeline prior to neural network ingestion. This preprocessing module systematically executed three distinct operations. First, the system performed noise reduction by programmatically discarding non-mathematical administrative columns to optimize tensor processing. Following this, the module executed JSON deserialization to unpack nested arrays within the annotations and text_annotations columns, thereby extracting precise bounding box geometries (bbox) and baseline semantic identities (supercategory). Finally, the system performed saliency mapping (as illustrated in Figure 1) and extracted the layout's color centroids. Each layout was summarized by a

single saliency weight, computed from the SIFT keypoint density of its image, which replaced the need to feed computationally heavy, raw spatial matrices directly into the GNN.

The culmination of this preprocessing pipeline yielded a structurally filtered node array, formatted precisely to populate the GNN's initial feature tensor ($h^{(0)}$). A representative sample of this structured training dataset, demonstrating both structural boundaries and extracted algorithmic features, is detailed in Appendix A.

Empirical Master-Layout Dataset

To resolve the domain transfer gap observed when applying layout models trained on generalized e-commerce posters to authentic promotional graphics, an empirical dataset comprising 200 authentic graphic design master layouts was curated as a specialized research instrument. In this framework, a master layout represented any unflattened or rasterized visual composition containing institutional or event branding assets uploaded to initialize orchestration. While the CGL-Dataset V2 established baseline spatial intelligence, preliminary evaluations indicated that detectors trained exclusively on generalized e-commerce banners suffered from significant confidence degradation when deployed on real institutional event designs. Consequently, this curated domain-specific dataset served as the formal training and validation corpus for the Semantic Layout Parsing module.

The dataset encompassed 200 distinct, high-resolution master layout posters spanning academic, cultural, and organizational event domains. These graphical assets were collected from official institutional publication repositories and public campus organization channels under academic fair-use guidelines.

All 200 layout images were annotated manually in Roboflow. For consistency across the pipeline, the annotations adopted the standardized four-class schema established by Zhou, Xu, Ma, Ge, Jiang, and Xu [2022]: logo, text, underlay, and embellishment. The bounding boxes were drawn under written class definitions and boundary rules. To minimize annotator subjectivity, two of the researchers annotated the images, and the third

researcher, as the group leader, reviewed their annotations against the written class definitions.

The annotated corpus was partitioned using an 80/20 train-to-validation split, allocating 160 images to the training set and 40 images to the held-out validation set. Rigorous programmatic preprocessing and quality filtering were executed prior to model ingestion. A dedicated normalization pipeline standardized coordinate definitions to resolve mixed polygon and bounding box geometry exports. Furthermore, an area-threshold filter programmatically removed outlier bounding boxes whose spatial footprints exceeded 50% of the total canvas area, preventing ambiguous background containers from being learned as layout elements. Finally, an independent graphic design expert reviewed a sample of 15 annotated images from the first 124-image batch against the written class definitions on September 29, 2026. The expert found the boxes correctly placed on all 15 images and correctly categorized on 10; on the other five, parts of logos or icons had been labeled as embellishments or grouped with adjacent text, and the overall assessment was "acceptable with minor issues noted." The dataset was subsequently expanded to 200 images and re-annotated.

The resulting curated dataset was utilized directly to train the deployed layout detector (a YOLOv8s architecture initialized from COCO pretrained checkpoints and trained at 1024-pixel resolution). This dedicated dataset enabled the perceptual parsing stage to extract structural bounding boxes from authentic master layouts with substantially higher confidence prior to graph construction and constraint solving.

To adapt spatial coordinates across heterogeneous formats, the system mapped target elements to the master layout based on shared semantic roles (e.g., pinning the logo row and header, while omitting incompatible positional data such as a poster's title on an academic certificate). Target elements with matching roles were pinned to their original coordinate anchors from the master layout, and all remaining unmapped elements were positioned sequentially between them via linear interpolation. Following the Graph Neural Network's



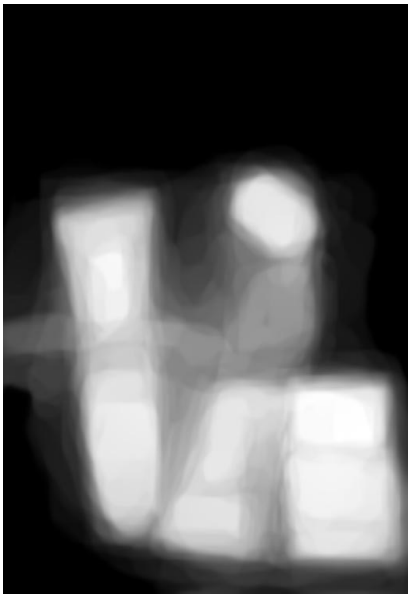
(a) Original annotated layout



(b) Inpainted background isolating the canvas



(c) Saliency map output



(d) Saliency map utilized for preprocessing

Figure 1: **CGL-Dataset Visual Extraction Pipeline.**
(a) The original annotated layout. (b) The inpainted background isolating the canvas. (c & d) The corresponding saliency maps utilized by the preprocessing module to calculate the $P_{legibility}$ visual noise penalty weight prior to GNN ingestion.

spatial refinement, the generated output was registered back onto these exact anchors, ensuring the final layout strictly preserved the master design's foundational structure.

While the offline dataset established the baseline spatial intelligence, the framework also required a secondary, dynamic data stream during runtime orchestration. Because the framework was specifically designed to handle distinct administrative domains—such as event identification cards and Academic Certificates—the system was required to ingest the user's structural requirements prior to final coordinate calculation.

Rather than injecting hardcoded data records at this stage, the system ingested a structural schema. This lightweight JSON payload defined the expected structural classes of the variables (e.g., text nodes or repeating geometric groups). Crucially, to align with the GNN's implicit geometric hierarchy, the schema explicitly assigned a Spatial Hierarchy Index to each variable. The Spatial Hierarchy Index instructed the Cassowary constraint solver to arrange nodes sequentially based on their predefined reading order (e.g., spanning from index 0 for the logo row down to index 8 for signature blocks on a certificate, or index 0 to 5 on an event ID). Rather than functioning as a direct size ranking, the index established each element's semantic role and vertical stack position. The system subsequently assigned a fixed baseline bounding box size to each specific role, ensuring that critical data points—such as the event title and attendee name—received the largest geometric dimensions on the event ID, regardless of their sequential placement. A sample representation of this runtime schema payload, utilizing an academic certificate context, is shown in Appendix B.

To guarantee deterministic execution and mathematical rigor, both the preprocessed dataset arrays and the dynamic payload structures were explicitly mapped to the algorithmic variables established within the system architecture. Table 1 demonstrated this precise correlation.

Synthetic Administrative Templates and Vector Assets

To rigorously evaluate the framework's real-time orchestration engine, a controlled dataset of synthetic administrative templates was engineered. Rather than utilizing abstract

Table 1
Data-to-Algorithm Parameter Mapping

Ingested Data Field	Algorithmic Application	Mathematical Variable	Processing Module
`bbox [x, y]`	Top-left spatial coordinate	(x_i, y_i)	GNN Initial Feature Encoding
`bbox [w, h]`	Baseline geometric tensor	(w_i, h_i)	Implicit Geometric Hierarchy
`extracted_color_centroids`	Algorithmic palette mapping	c_i	Node Feature Aggregation
`administrative_typography (font_family, weight)`	Strict brand typographic compliance	τ	Pre-Orchestration Schema Validation
`extracted_saliency_weight`	Visual noise penalty metric	$S(p)$	Heuristic Optimization Loop
`spatial_hierarchy`	Tensor allocation priority	Placeholder Tensor	Dynamic Node Instantiation

graphic structures, these master files were explicitly modeled to represent high-constraint institutional domains, specifically Academic Certificates and event identification cards. The event identification cards were structured in an A6 vertical proportion and contained standardized fields including a logo row, an event title, an attendee name, and a designated role. These templates were structured as interconnected Scalable Vector Graphics (SVG) and JSON schemas, designed to encompass varying degrees of spatial complexity.

Crucially, these synthetic assets acted as the primary testing instruments to stress-test the Cassowary constraint solver. The synthetic payloads were filled with extreme content: every text field at 25%, 50%, 75%, and 100% of its maximum length, both as real words and as one unbroken string; each field alone at its maximum; each field one character over its maximum; and the certificate's repeating signature blocks increased from 1 to 10. Each output was checked for its structural accuracy (the F1-score defined under Evaluation Metrics), for whether the elements kept their spatial hierarchy order, and for whether every

rendered line of text fit inside its bounding box, measured with the actual glyph widths of the font. The results of this test are reported in Chapter 4.

Research Procedure

The algorithmic pipeline was executed in four distinct developmental phases, mapping directly to the proposed tri-layer orchestration architecture. This procedural approach ensured the systematic integration of the perceptual extraction models, the real-time constraint solvers, and the quantitative evaluation metrics.

Phase 1: Data Extraction and Graph Construction

In the initial phase, the Deterministic Style Extraction modules were programmed to autonomously parse master design templates. The system utilized K-Means Clustering and the SIFT algorithm to mathematically isolate dominant color centroids and scale-invariant stylistic markers. Concurrently, the system executed Semantic Layout Parsing utilizing a YOLOv8s architecture for object bounding box detection—initialized from COCO pre-trained weights and trained at a 1024-pixel resolution exclusively on the empirical master-layout dataset—alongside Tesseract OCR, which labelled the detected text regions, converting the flattened raster pixels into structured, editable nodes; the words placed in the generated layouts came from the user's validated content payload. In parallel, the Graph Neural Network (GNN) architecture was constructed and trained using the CGL-Dataset. The GNN was trained to map these extracted visual elements into a formal mathematical graph, establishing the hierarchical node-edge dependencies and spatial reasoning baselines required for the layout generation.

This interface (Figure 2) demonstrated the initial data extraction and administrative configuration phase prior to layout synthesis, the first of the dashboard's three steps, which a step bar along the top marked. The left side displayed the uploaded master layout, with the autonomous algorithmic extraction beneath it: the dominant color centroids identified

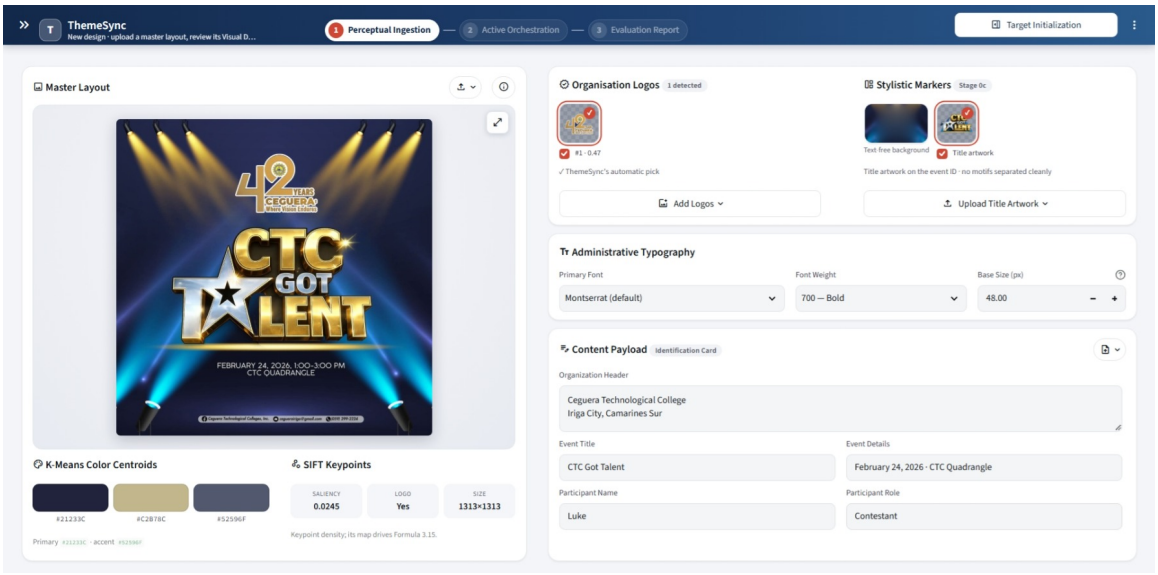


Figure 2: **Perceptual Ingestion and Initialization Dashboard.**

via K-Means clustering, and the SIFT keypoint saliency together with whether a logo was detected. The elements parsed by the layout detector could be opened over the preview as a relational node graph. The right side presented the rest of the ”Hybrid Visual DNA” in cards: the detected organization logos and the Stage 0c stylistic markers (the text-free background and the title artwork), each of which the designer could keep or remove, with uploads for missed logos and a replacement title artwork; the administrative typography, where the user deterministically injected the font family, font weight, and baseline size; and the content payload for the target format. The target format was chosen in a Target Initialization panel opened from the top bar, which also held the command that initialized the orchestration. By allowing the designer to review the machine-extracted variables, define their strict typographic constraints, and select the target format on a single workspace, the framework ensured that all necessary constraints and spatial boundaries were established before initializing the orchestration engine.

Phase 2: Constraint Satisfaction Problem (CSP) Integration and State Synchronization

Following the construction of the relational graph, the second phase focused on real-time reactive layout synthesis. Prior to spatial calculation, the framework executed Dynamic Node Instantiation. The system ingested the user's Content Payload (external administrative datasets formatted as JSON) and algorithmically assigned Baseline Geometric Tensors to the new variables, establishing their Implicit Geometric Hierarchy within the graph. The Cassowary Algorithm was then mathematically implemented within the orchestration layer to act as the primary linear arithmetic constraint solver. Constraint equations were formulated and embedded to ensure bounding boxes dynamically adapted to varying aspect ratios without spatial overlap. Simultaneously, the Bi-directional State Machine was coded to serialize ongoing project states into lightweight JSON formats. This synchronization layer was explicitly programmed to capture manual user interventions from the Human-in-the-Loop (HITL) interface, translating them into hardened mathematical constraints to prevent destructive overwriting during later re-solves; every earlier state stayed in the chain and could be restored.

Phase 3: Heuristic Testing and Optimization

The third procedural phase involved the rigorous testing and refinement of the generated coordinate layouts. The system was subjected to both the synthetic administrative templates and a testing battery of real, unseen event posters to execute parallel state-space explorations, effectively generating multiple layout candidates. The inclusion of authentic posters during this phase was strictly necessary because the calculation of the Legibility Penalty required a genuine visual background to accurately measure high-frequency visual noise and saliency. Specifically, these parallel explorations generated distinct candidates by varying four core compositional parameters: the structural layout archetype (e.g., classic centered, left-aligned, art column, or floating card), the page margin, the vertical rhythm

governing the minimum gaps between stacked elements, and the decorative motif arrangement (such as diagonal, four corners, or a single enlarged hero motif). These algorithmic outputs were mathematically evaluated against the established heuristic Fitness Function.

While the heuristic fitness function mathematically scored every generated candidate, the final recommended output was not determined solely by the lowest absolute F_{layout} penalty. Instead, the master layout's original visual composition dictated the target layout archetype (e.g., left-aligned text, an artwork column, or a footer band, with a classic centered structure when the master showed none of these cues). Without a master layout, the candidate with the lowest F_{layout} was selected. The fitness function subsequently ranked the candidates within that mandated archetype to select the optimal output, ensuring the generation respected the source design's structural intent.

The system's penalty weights (for the collision, alignment, whitespace, and legibility penalties, the last derived from cross-referencing text coordinates against the SIFT Saliency Map) were iteratively calibrated. This optimization loop continued until the mathematical scores demonstrated that the generated coordinate predictions consistently aligned with strict architectural constraints and human-centric design rules. Once the spatial coordinates were heuristically validated, the Aesthetic Rendering Pipeline executed. This final aesthetic rendering pipeline projected not only the K-Means color clusters and extracted stylistic typography onto the generated bounding boxes, but also the reconstructed background layer, the isolated decorative motifs, and the title artwork, thereby producing a fully populated and visually harmonized graphic layout.

This unified workspace (Figure 3) illustrated the real-time execution of the Cassowary constraint solver, the schema validation protocols, and the aesthetic evaluation algorithms. The central canvas displayed the design adapted to the target format and was edited in place: the designer selected, moved, resized, locked, or retyped an element directly on it, and each completed move, resize, or lock triggered one re-solve. Beneath the canvas, the Recommendation Engine displayed the generated candidates as a strip of thumbnails ranked by

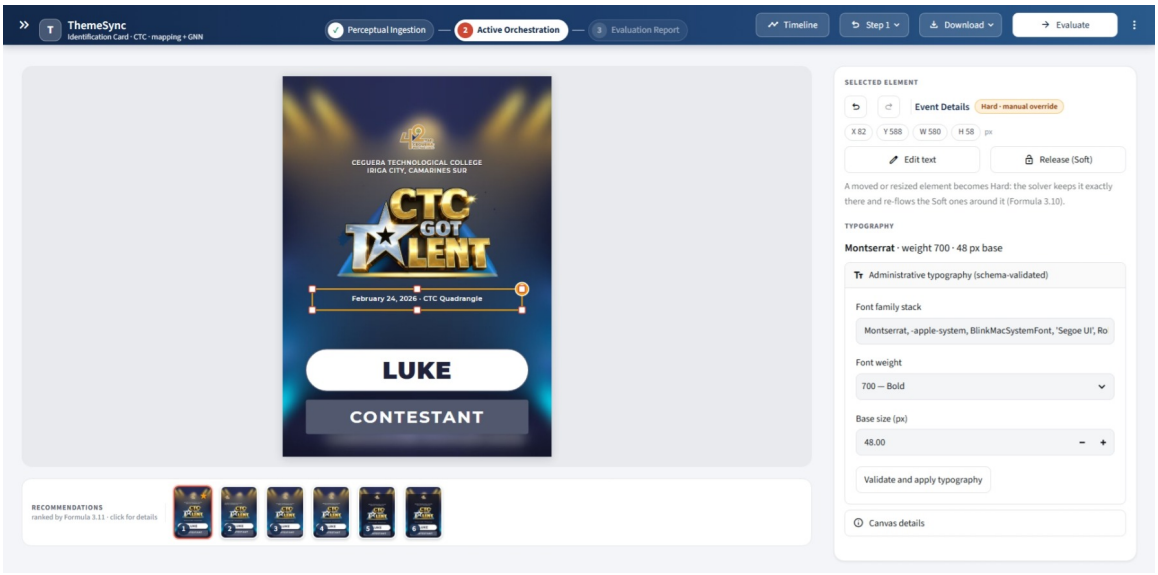


Figure 3: **Active Orchestration and Heuristic Scoring Workspace**

the heuristic fitness function, where the underlying master layout's composition selected the appropriate layout archetype and the fitness function ranked the variations within it. The right-hand panel exposed the selected element's constraint state: its spatial coordinates (X, Y, W, H), undo and redo, a control for editing its text, and the choice between processing it as an algorithmic "Soft" constraint or a locked "Hard" constraint, followed by the typographic parameters, which were validated against the schema before they were applied. A State Timeline, opened from the top bar, displayed the serialized JSON chain of the Bi-directional State Machine, recording every state and the non-destructive retention of manual user refinements.

The dashboard workflow formally concluded with a third step: a comprehensive Evaluation Report generated for the finalized layout (Figure 4). This report detailed the heuristic fitness calculation (Formula 3.11) alongside the final recommendation, the structural accuracy (F1 score) per element, the manual edits retained through subsequent re-solves, stage and per-edit latency metrics, the K-Means elbow graph, and the schema validations.

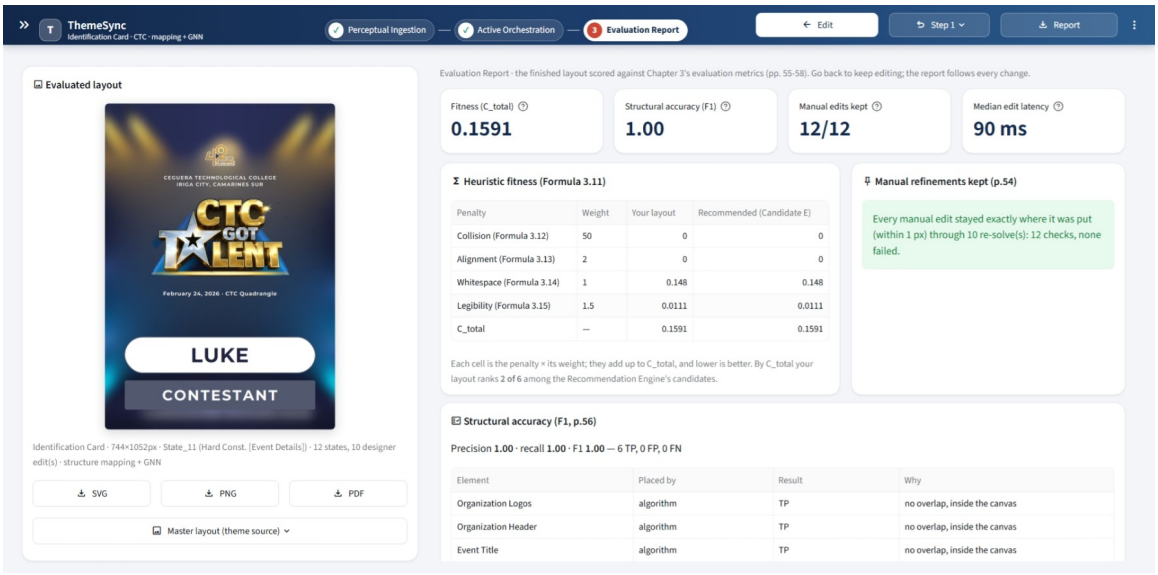


Figure 4: **Evaluation Report**

Phase 4: Expert Aesthetic Validation

Following the algorithmic generation and mathematical validation of the spatial coordinates in Phase 3, the orchestration framework underwent a final human-in-the-loop expert evaluation to assess its aesthetic quality against industry standards. A panel of four designated domain experts—defined as professional layout artists and graphic designers with documented industry experience—evaluated the ThemeSync outputs. The deployment of multiple evaluators followed established usability and layout evaluation practice: Nielsen and Molich [1990] found that individual evaluators identified only 20–51% of usability problems, whereas aggregated evaluations from as few as three to five evaluators performed well, and the studies that introduced the CGL-Dataset rated generated layouts with panels of professional and novice designers [34, 17]. The experts were tasked with reviewing a specifically designed experimental sample comprising 64 generated layouts. This evaluation set was derived from 16 master layouts selected by purposive sampling [4] under three criteria: each was a real event poster from a school organization; none appeared in any of the system's training data, which was verified with an image-duplicate

test; and its text fit the content fields of both target formats. Each master layout was orchestrated across two target formats (Academic Certificates and event identification cards) under two distinct experimental conditions. To rigorously evaluate the framework's spatial intelligence, a blind baseline condition was introduced. Layouts generated by the full ThemeSync orchestration engine were compared against baseline outputs generated with identical content but with both the master-structure mapping and the GNN module deactivated, so that the baseline arranged the elements as a naive reading-order stack. The two conditions were therefore labeled *structure mapping + GNN* and *baseline*. Both conditions shared the layout archetype selected from the master's composition, the extracted palette, the reconstructed background, and the stylistic markers.

The resulting 64 layouts were compiled into printed evaluation packets, with each page displaying the original master layout alongside a generated candidate. To prevent evaluation bias, the generated layouts were assigned random sample codes, the page order was independently randomized for each expert, and pages derived from the same master layout were never placed adjacently. This evaluation sample size aligned with the established validation protocols of Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] and Li, Liu, Feng, Zhu, Li, Zhang, Lv, Zhu, Shen, Lin, and Shao [2023], which utilized 60-image test sets to validate generative layout models.

Rather than modifying the underlying computational architecture, the experts strictly evaluated the final visual outputs. They utilized a standardized Mean Opinion Score (MOS) rubric to quantitatively grade each generated layout. This evaluation measured critical design parameters that extended beyond absolute mathematical bounds, bridging the gap between computational constraint satisfaction and subjective human design validity.

Evaluation Metrics

The validation of the orchestration framework utilized a mixed-methods approach. The core system architecture relied on controlled, quantitative metrics to measure algorithmic

precision, detection accuracy, and computational efficiency. However, to ensure that the mathematically optimized layouts translated into viable professional designs, this rigorous computational benchmarking was followed by a structured, quantifiable evaluation by a panel of four human domain experts. This dual-validation methodology was designed to assess both the framework's mathematical constraint satisfaction and its real-world aesthetic validity.

Intersection over Union (IoU)

Intersection over Union (IoU) was strictly confined to Phase 1 of the methodology to evaluate the extraction accuracy of the Semantic Layout Parsing module. During the initial ingestion of the Master Template, this metric calculated the area of overlap between the predicted bounding boxes generated by the YOLOv8 object detection model and the ground-truth annotations established in the empirical master-layout dataset's held-out validation split (40 images). By dividing the area of overlap by the area of union, the system yielded a strict, quantifiable score defining the exact geometric fidelity of the extracted nodes prior to relational mapping. Crucially, IoU was not utilized to evaluate the final dynamically generated layouts in Phase 3 or Phase 4, as orchestrating nodes across heterogeneous target aspect ratios inherently precluded the existence of a static ground-truth coordinate map.

Algorithmic Latency

This metric measured the time complexity and execution speed of the underlying algorithms, particularly the Cassowary Algorithm and the Bi-directional State Machine. Algorithmic latency was measured in milliseconds over recorded sequences of designer edits and judged against two response-time limits required for real-time Human-in-the-Loop (HITL) interface responsiveness [21]: the 0.1-second limit for a response perceived as instantaneous, applied to each constraint recalculation by the Cassowary solver, and the

1.0-second limit for uninterrupted task flow, applied to each full state synchronization (re-solving, re-scoring, and re-rendering the layout and saving the state chain).

Structural Accuracy Rate (F1-Score and Classification Metrics)

The overall reliability of the reactive layout synthesis engine (Phases 2 and 3) was scored utilizing an F1-Score, derived from the harmonic mean of precision and recall. Because generating layouts for heterogeneous aspect ratios precluded the existence of a ground truth, the classification parameters for this metric were strictly mathematically defined by Constraint Satisfaction rather than spatial overlap.

A True Positive (TP) was recorded when the algorithm successfully generated and positioned bounding box coordinates for a required semantic node, provided that the node mathematically satisfied all designated constraints (i.e., it triggered zero collision penalties and strictly respected user-defined Hard Constraints).

Conversely, a False Positive (FP) was registered if the algorithm synthesized bounding box coordinates that violated a mathematically defined spatial boundary. This included invoking a collision penalty by overlapping another element (including a designer-locked one), or protruding beyond the absolute dimensions of the target viewport.

Finally, a False Negative (FN) occurred when the algorithm completely failed to generate bounding box coordinates for a semantic node that was mathematically mandated by the Dynamic Content Payload, resulting in a missing structural element on the final design canvas.

The Heuristic Fitness Function

Layout evaluation was framed strictly as a mathematical optimization problem aiming to minimize the aggregate penalty score defined in Formula 3.11. The heuristic fitness function objectively scored parallel layout candidates by computing the Collision Penalty (Formula 3.12 to calculate overlapping nodal areas), the Alignment Penalty (Formula 3.13

to measure distances from established gridlines), the Whitespace Penalty (Formula 3.14 to evaluate the mathematical ratio of negative space), and the Legibility Penalty (Formula 3.15 to quantify visual interference by cross-referencing text coordinates against the high-frequency keypoints of the SIFT Saliency Map). By utilizing these explicit mathematical thresholds, the framework translated selected aesthetic principles into machine-readable metrics, so that structural accuracy could be validated without subjective human surveying.

Mean Opinion Score (MOS) for Aesthetic Alignment

To execute the expert aesthetic validation in Phase 4, the designated layout experts evaluated the generated multi-format layouts using a structured five-point Likert-type rating scale [18]. This Mean Opinion Score (MOS) quantified the subjective aesthetic validity of the final designs into rigorous, measurable data. The panel of experts evaluated the 64 generated layouts under both the structure mapping + GNN and the blind baseline conditions against three core design parameters:

1. **Visual Hierarchy Preservation:** Evaluated whether the algorithmic coordinate placement maintained the correct structural dominance (e.g., ensuring a student's name or a certificate title visually remained the primary focal point relative to the surrounding nodes), serving as the subjective measure of visual emphasis. [23]

2. **Brand and Typographic Consistency:** Evaluated the framework's success in enforcing the Hybrid Visual DNA, specifically grading the accurate rendering of user-defined fonts, baseline geometric sizes, and algorithmically extracted color palettes without hallucination to ensure brand coherence. [29]

3. **Constraint and Whitespace Harmony:** Evaluated the subjective quality of the spatial arrangement, confirming the visual absence of element occlusion (overlapping text/graphics) and verifying that the layout's negative space aligned with professional composition standard. [23, 34]

Each of these qualitative parameters functioned as the human-evaluated counterpart to the system's internal computational metrics. While the implicit geometric hierarchy, the Visual DNA schema, and the heuristic fitness function (specifically Formulas 3.12 and 3.14) scored the layouts mathematically, the MOS evaluation tested whether these computational bounds translated into perceptually valid and aesthetically pleasing designs.

For each parameter, the experts assigned a numerical score based on the MOS rating scale shown in Table 2. This rubric was adapted from the standard ITU-T P.800 Absolute

Table 2
Mean Opinion Score Rating Scale for Aesthetic Alignment

Score	Rating	Description
5	Excellent	No noticeable flaws
4	Good	Minor aesthetic deviations
3	Acceptable	Noticeable but non-destructive deviations
2	Poor	Significant layout fragility
1	Fail	Severe structural overlap or typographic failure

Category Rating scale [8]. To align with professional design evaluations, the standard midpoint "Fair" was renamed to "Acceptable," the baseline "Bad" was renamed to "Fail," and design-specific descriptive anchors were added to contextualize the grading criteria.

Statistical Treatment

The statistical treatment was fixed before any rating was examined: the analysis was implemented as a script and committed to the project's version control before the experts' score sheets were imported, and it was then run unchanged. The following statistical tools and procedures were employed:

1. **Descriptive Statistics for Aesthetic Alignment:** In accordance with ITU-T Recommendation P.800 [8], the Mean Opinion Score (MOS) was computed as the arithmetic mean of the ratings, together with the standard deviation, for each of the three parameters and for the overall score (the mean of the three parameters), separately for

each condition and each format. The number of ratings given at each point of the five-point scale was also reported.

2. **Inter-Rater Agreement:** The agreement of the four experts was measured with Kendall's coefficient of concordance (W) [11], corrected for ties, for each parameter and overall across the 64 layouts, with its chi-square test of significance. Kendall's W compares how the experts order the layouts, which suits the ordinal ratings. It was interpreted on the scale of Schmidt [1997], on which values from 0.5 to 0.7 indicate moderate agreement and values above 0.7 indicate strong agreement.

3. **Comparison of the Conditions:** Each layout's score was the mean of its four experts' ratings. The structure mapping + GNN and baseline layouts generated from the same master layout in the same format formed a pair, giving 32 pairs, which were compared with the two-sided Wilcoxon signed-rank test [30]; pairs with equal scores were excluded from the ranking. Both formats were tested together, and the MOS of each format was reported without a separate test. The overall MOS was the main test, named in advance; the three parameters were follow-up tests, and the p-values of the four tests were adjusted with the Holm correction [6]. The results were judged at a significance level of 0.05.

4. **Effect Size:** Every p-value was reported with the matched-pairs rank-biserial correlation r [12], which ranges from -1 to $+1$, and its magnitude was described with the conventional thresholds of 0.1, 0.3, and 0.5 for small, medium, and large effects [3]. Because 32 pairs could miss a small difference, a non-significant result was reported as "no significant difference was detected" rather than as equality.

5. **Sensitivity Analyses:** As secondary checks, the main tests were repeated with the paired t-test, with the Bonferroni correction in place of the Holm correction, and without the one pair whose two layouts rendered identically. The absolute agreement

of the experts was also estimated with the intraclass correlation coefficient ICC(2,1) [28] and Krippendorff's α [14].

6. **System Metrics:** For the Phase 3 metrics, the fitness score (F_{layout}, Formula 3.11), the latency, the F1-score, and the IoU were summarized by their mean and standard deviation, and the fitness scores of the two conditions were compared with the same Wilcoxon signed-rank test on the same 32 pairs.

Theoretical and Conceptual Framework

This section established the foundational theoretical principles and the overarching conceptual paradigm that governed the architecture and evaluation of the ThemeSync orchestration framework. It delineated the core computational theories underpinning the system, specifically drawing upon constraint-based layout theory to justify the mathematical resolution of spatial boundaries, alongside computational aesthetic theories to ground the heuristic and expert evaluation of the layouts. Furthermore, this section presented the conceptual framework, explicitly mapping the systematic interaction of variables throughout the execution pipeline. It illustrated the continuous data flow from the initial ingestion of unstructured graphical layouts and user-defined parameters, through the concurrent processing of the neural network and mathematical modules, to the final output of stabilized, compositionally aligned design candidates. By synthesizing these theoretical and conceptual constructs, this section provided the rigorous academic scaffolding required to mathematically and logically justify the system's structural design and subsequent empirical methodology.

Figure 5 illustrated the theoretical framework that underpinned this research. The ThemeSync framework was grounded in five fundamental computer science theories: Graph Theory, the Constraint Satisfaction Problem (CSP) paradigm, Automata Theory, Human-Computer Interaction Theory, and Optimization Theory. These interconnected theoretical

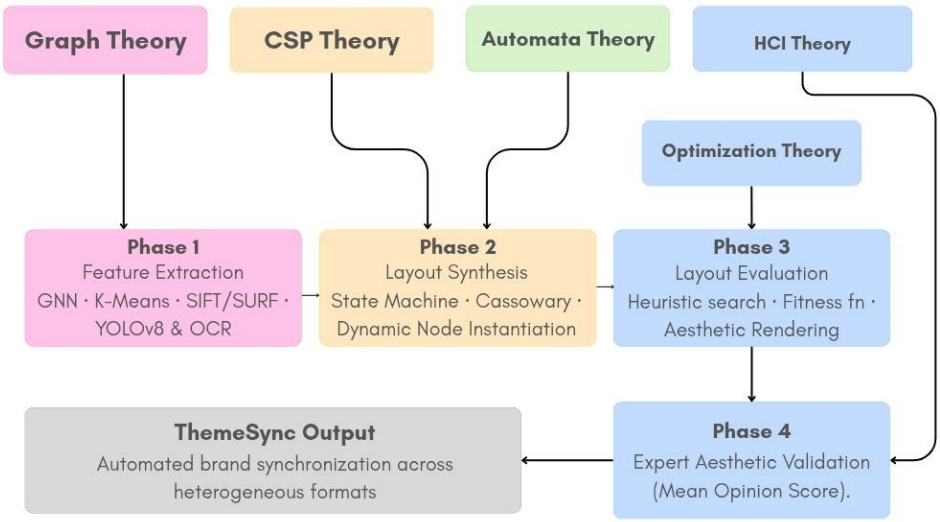


Figure 5: **Theoretical Framework Diagram**

models provided the formal mathematical logic required to transition graphic layout orchestration from a static, coordinate-based generation task into a dynamic, stateful algorithmic pipeline.

Graph Theory

Graph Theory served as the primary theoretical basis for Phase 1 of the system, governing the relational logic of the layout components. In this context, a graphic layout was not modeled as a flattened array of pixels, but rather as a highly structured mathematical graph. Individual design elements, such as text blocks, image containers, and background shapes, operated as nodes, while the spatial and structural dependencies between them—such as proximity, alignment, and hierarchical nesting—operated as edges. This theory justified the application of Graph Neural Networks (GNN) and configurable graph-based frameworks within the system, establishing that deep learning models could accurately parse, traverse, and reconstruct the topological anatomy of a design by mathematically evaluating these node-edge relationships prior to rendering [31]. Furthermore, integrating percep-

tual extraction techniques, such as algorithmic palette mapping, directly into this graph ensured that stylistic and color identities remained mathematically consistent during structural transformations [7].

Constraint Satisfaction Problem (CSP) Paradigm

The CSP paradigm underpinned Phase 2 by providing the theoretical basis for reactive layout synthesis. CSP defined problems where the state of a set of objects had to satisfy a specific number of logical or arithmetic constraints. Applied to spatial orchestration, responsive layout generation transformed into a complex equation-solving environment capable of supporting real-time GUI interactions [20]. Viewport dimensions and target aspect ratios were treated as mathematical variables, while structural rules and user intents were encoded as linear inequalities and adaptive OR-constraints [9]. This foundation governed the implementation of constraint-based layout generation models [10, 32], proving that dynamic spatial harmony could be mathematically enforced in real time — eliminating the need for stochastic pixel placement.

Automata Theory

To address the persistent issue of destructive layout generation, the framework integrated principles from Automata Theory and state-machine logic. Automata Theory is the study of abstract machines and systems that transition between distinct computational states based on defined inputs. Within ThemeSync, this grounded the Bi-directional State Machine and its corresponding JSON schema handling. It provided the formal logical proof that the orchestration engine could maintain a persistent ”memory” of localized manual edits—treating a user’s manual refinement as a highly specific computational state—and seamlessly transition the global template structure to a new iteration without irreversibly overwriting the human-in-the-loop modifications.

Optimization Theory

Optimization Theory supported Phase 3 of the system, governing the algorithmic evaluation of the generated layouts. Optimization Theory focuses on finding the best available mathematical solution from a massive set of feasible, parallel alternatives. In the context of automated orchestration, determining the optimal layout required navigating a rigorous mathematical search space. This theory justified the system's Heuristic Search mechanism, establishing that subjective aesthetic composition rules—such as geometric balance, whitespace distribution, and margin collision avoidance—could be quantified into objective penalty scores. By minimizing these penalty scores via a mathematical Fitness Function, the system sought to bridge the gap between automated spatial calculation and human-centric aesthetic validity.

Human-Computer Interaction (HCI) Theory

Finally, to support Phase 4, the framework drew upon Human-Computer Interaction (HCI) theory, which asserts that computational systems must align with human cognitive and perceptual standards to achieve practical utility. While Optimization Theory governed the mathematical minimization of spatial errors, HCI theory justified the necessity of the Mean Opinion Score (MOS). It provided the theoretical grounding that pure mathematical constraint satisfaction was insufficient for creative orchestration without human-in-the-loop validation. By structuring the experts' subjective aesthetic evaluation into a quantifiable metric, the study could measure how far algorithmic layout generation met human-centric design expectations.

Notes

[1] Greg J. Badros, Alan Borning, and Peter J. Stuckey. 2001. The Cassowary linear arithmetic constraint solving algorithm. *ACM Transactions on Computer-Human Interaction*, 8, 4, 267–306. DOI: 10.1145/504704.504705.

[2] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. 2023. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. MIT Press. DOI: 10.48550/arXiv.2104.13478.

[3] Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences*. (2nd ed.). Lawrence Erlbaum Associates, Hillsdale, NJ, USA. Retrieved October 12, 2026 from https://books.google.com/books/about/Statistical_Power_Analysis_for_the_Behav.html?id=2v9zDAsLvA0C.

[4] Ilker Etikan, Sulaiman Abubakar Musa, and Rukayya Sunusi Alkassim. 2016. Comparison of convenience sampling and purposive sampling. *American Journal of Theoretical and Applied Statistics*, 5, 1, 1–4. DOI: 10.11648/j.ajtas.20160501.11.

[5] Miguel Garcia and Jia-Ray Chen. 2023. New approach to dominant color extraction in images with wide range of hues using CIE lab space. *Technologies*, 11, 6, 230. DOI: 10.3390/technologies11060230.

[6] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6, 2, 65–70. DOI: 10.2307/4615733.

[7] Haoran Huang, Yong Chen, and Zhaolin Liu. 2025. Recoloring cartoon images based on palette mapping using K-means. *Engineering Proceedings*, 10, 7, 82. DOI: 10.3390/engproc2025107082.

[8] ITU-T. 1996. Methods for subjective determination of transmission quality. ITU-T Recommendation P.800. International Telecommunication Union, Geneva, Switzerland. Retrieved October 12, 2026 from https://www.itu.int/rec/t-rec-p.800.

[9] Yue Jiang, Wolfgang Stuerzlinger, Matthias Zwicker, and Christof Lutteroth. 2020. ORCSolver: an efficient solver for adaptive GUI layout with OR-constraints. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '20)*. ACM, 1–14. DOI: 10.1145/3313831.3376610.

[10] Zhaoyun Jiang, Jiaqi Guo, Shizhao Sun, Huayu Deng, Zhongkai Wu, Vuksan Mijovic, Zijiang James Yang, Jian-Guang Lou, and Dongmei Zhang. 2023. Layoutformer++: conditional graphic layout generation via constraint serialization and decoding space restriction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2023)*. DOI: 10.48550/arXiv.2208.08037.

[11] M. G. Kendall and B. Babington Smith. 1939. The problem of m rankings. *The Annals of Mathematical Statistics*, 10, 3, 275–287. Retrieved October 12, 2026 from https://doi.org/10.1214/aoms/1177732186.

[12] Dave S. Kerby. 2014. The simple difference formula: an approach to teaching nonparametric correlation. *Comprehensive Psychology*, 3, 11.IT.3.1. DOI: 10.2466/11.IT.3.1.

[13] Martin Kleppmann. 2022. Making CRDTs Byzantine fault tolerant. In *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '22)*. Association for Computing Machinery. DOI: 10.1145/3517209.3524042.

[14] Klaus Krippendorff. 2004. *Content Analysis: An Introduction to Its Methodology*. (2nd ed.). Sage Publications, Thousand Oaks, CA, USA. Retrieved October 12, 2026 from https://books.google.com/books/about/Content_Analysis.html?id=q657o3M3C8cC.

[15] Tanakorn Lertrusdachakul, Phayung Meesad, and Sunantha Kittirachakul. 2025. Enhancing clustering performance: a hybrid generalized K-means approach. *Interna-*

tional Journal of Advanced Research (IJAR), 13, 1. Retrieved March 3, 2026 from https://www.journalijar.com/uploads/2025/04/6821b3f719224_IJAR-50827.pdf.

[16] Bohan Li, Dawei Li, Ming Fu, and Shaowei Cai. 2024. SMT-layout: a MaxSMT-based approach supporting real-time interaction of real-world GUI layout. *arXiv preprint arXiv:2411.12271*. Retrieved March 3, 2026 from https://arxiv.org/abs/2411.12271.

[17] Fengheng Li, An Liu, Wei Feng, Honghe Zhu, Yaoyu Li, Zheng Zhang, Jingjing Lv, Xin Zhu, Junjie Shen, Zhangang Lin, and Jingping Shao. 2023. Relation-aware diffusion model for controllable poster layout generation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 1249–1258. DOI: 10.1145/3583780.3615028.

[18] Rensis Likert. 1932. A technique for the measurement of attitudes. *Archives of Psychology*, 22, 140, 1–55. Retrieved October 12, 2026 from https://archive.org/details/likert-1932.

[19] Yang Liu, Yan Liu, and Yan Li. 2026. Computational frameworks for interactive multimedia art installations in public spaces using real time digital control and visualization. *PeerJ Computer Science*, 12, e3384. DOI: 10.7717/peerj-cs.3384.

[20] Daniel Lukes, John Sarracino, Cedric Coleman, Hila Peleg, Sorin Lerner, and Nadia Polikarpova. 2021. Synthesis of web layouts from examples. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. DOI: 10.1145/3468264.3468533.

[21] Jakob Nielsen. 1993. *Usability Engineering*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 135. Retrieved October 12, 2026 from https://archive.org/details/usabilityenginee0000niel.

[22] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)*. Association for Computing Machinery, New York, NY, USA, 249–256. DOI: 10.1145/97243.97281.

[23] Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. 2014. Learning layouts for single-page graphic designs. *IEEE Transactions on Visualization and Computer Graphics*, 20, 8, 1200–1213. DOI: 10.1109/TVCG.2014.48.

[24] Roberto Penco, Damir Pintar, Mihaela Vranić, and Marko Šoštarić. 2025. Large language model-driven framework for automated constraint model generation in configuration problems. *Applied Sciences*, 15, 12, 6518. DOI: 10.3390/app15126518.

[25] Gheorghe Alin Sabău and Alin Claudiu Zamfiroiu. 2024. Real-time document collaboration—system architecture and design. *Applied Sciences*, 14, 18, 8356. DOI: 10.3390/app14188356.

[26] Roy C. Schmidt. 1997. Managing Delphi surveys using nonparametric statistical techniques. *Decision Sciences*, 28, 3, 763–774. Retrieved October 12, 2026 from https://doi.org/10.1111/j.1540-5915.1997.tb01330.x.

[27] M. Shaharom and K. Tahar. 2023. Multispectral image matching using SIFT and SURF algorithm: a review. *International Journal of Geoinformatics*, 19, 1, 13–21. Retrieved March 3, 2026 from https://ijg.e-geoinfo.com/index.php/journal/article/view/2495/1329.

[28] Patrick E. Shrout and Joseph L. Fleiss. 1979. Intraclass correlations: uses in assessing rater reliability. *Psychological Bulletin*, 86, 2, 420–428. DOI: 10.1037/0033-2909.86.2.420.

[29] Alina Wheeler. 2013. *Designing Brand Identity: An Essential Guide for the Whole Branding Team*. (4th ed.). John Wiley & Sons, Hoboken, NJ, USA, 29. Retrieved

October 12, 2026 from https://books.google.com/books/about/Designing_Brand_Identity.html?id=uSNHAAAAQBAJ.

[30] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1, 6, 80–83. DOI: 10.2307/3001968.

[31] Lingfei Wu, Peng Cui, Jian Pei, and Liang Zhao. 2022. *Graph Neural Networks: Foundations, Frontiers, and Applications*. Springer Nature. DOI: 10.1007/978-981-16-6054-2.

[32] Liwen Zhang, Sara Maqrot, Florent Mouysset, and Christophe Bortolaso. 2025. A constraint satisfaction problems based scalable framework to address large-scale realistic problems. In *Proceedings of the ICORES 2025*. SciTePress, 45–56. Retrieved March 3, 2026 from https://www.scitepress.org/Papers/2025/131096/131096.pdf.

[33] Jie Zhou, Ganqu Cui, and Shengding Hu. 2020. Graph neural networks: a review of methods and applications. *AI Open*, 1, 57–81. DOI: 10.1016/j.aiopen.2021.01.001.

[34] Min Zhou, Chenchen Xu, Ye Ma, Tiezheng Ge, Yuning Jiang, and Weiwei Xu. 2022. Composition-aware graphic layout gan for visual-textual presentation designs. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*. International Joint Conferences on Artificial Intelligence Organization, 4995–5001. DOI: 10.24963/ijcai.2022/692.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presented and discussed the results of the evaluation of ThemeSync, in one section for each specific objective, each starting with the data and conditions it used. Each table and figure was followed by its results and by a discussion of the principal findings, their interpretation, the related literature, the implications, the limitations, and a summary.

Relational Logic and Feature Extraction Model

The first specific objective was to design a Relational Logic and Feature Extraction model that integrated the Graph Neural Network (GNN), K-Means clustering, and SIFT to identify and map the structural dependencies and stylistic markers of master design files.

The deployed model used the YOLOv8s layout detector trained on the empirical master-layout dataset and the third version of the GNN. The detector was evaluated on the 40-image validation split of that dataset (607 annotated elements), and the GNN on 2,900 CGL-Dataset layouts held out from training, a seeded 5% of the training split. The transfer of the master's structure was measured on 16 unseen event posters chosen by purposive sampling, in both formats and three conditions: *structure mapping + GNN*, the full system, in which the target elements were mapped to the master's structure and refined by the GNN; *structure mapping only*, with the GNN off; and the *baseline*, which used neither and stacked the elements in reading order. All three shared the archetype, palette, background, and stylistic markers. The 16 posters, two formats, and three conditions gave the 96 runs of the Phase 3 battery.

Identification of Structural Elements (Layout Detector)

Table 3 presented the highest detection confidence of the first three detectors on three real event posters outside their training sets.

Table 3
Highest Detection Confidence on Three Real Posters Outside the Training Sets

Detector	La Serata	TAN-AW	LODT
v1 (CGL-Dataset only)	0.035 (0)	0.204 (0)	0.211 (0)
v2 (+ 65 master layouts)	0.740 (16)	0.735 (13)	0.694 (8)
v3 (+ 124 master layouts)	0.843 (10)	0.753[a] (9)	0.793 (11)

Values in parentheses are the number of detections at or above the 0.25 confidence threshold. [a]TAN-AW was part of v3's training data.

On the CGL-Dataset's own held-out images, v1 reached an mAP50 of 0.539. On the three event posters, its highest confidence was only 0.035 to 0.211, and no detection reached the system's 0.25 threshold. After fine-tuning on the first 65 annotated master layouts (v2), each poster produced 8 to 16 detections, with top confidences of 0.69 to 0.74. With 124 master layouts (v3), the posters produced 9 to 11 detections, with top confidences of 0.75 to 0.84.

The CGL-trained detector could not read event posters, and a little in-domain data fixed this; the confidence rose further with the larger batch. The CGL-Dataset's e-commerce posters [24] differ from school event posters in content and composition. dos Santos Júnior, Paixão, and Alvarez [2025] found YOLOv8 effective for the layout analysis of historical documents; here, it worked on event posters only after training on them. A public dataset from another domain cannot be assumed to transfer, so the study built its own. Only three posters were checked, and TAN-AW was part of v3's training data, so v3's value on it was not a fair test. Layout parsing required in-domain data.

The detector was then developed in two more steps (Table 4). Values were comparable within a panel but not across panels.

Table 4
Development of the Layout Detector

Detector	All	Logo	Text	Underlay	Embellishment
A. mAP50 on the validation labels of the 124-image batch					
v3 (YOLOv8n, 640 px)	0.363	0.620	0.584	0.037	0.212
v4 (YOLOv8s, 1024 px)	0.454	0.818	0.682	0.115	0.201
B. mAP50 on 32 validation images that neither model trained on					
v4 (124 images)	0.486	0.864	0.680	0.138	0.263
v5 (200 images)	0.728	0.977	0.684	0.643	0.609
C. Recall on the same 32 images					
v4 (124 images)	0.458	0.868	0.657	0.097	0.210
v5 (200 images)	0.671	0.938	0.722	0.516	0.507

Panels B and C left out the eight validation images that either model had trained on.

A larger YOLOv8s model trained at 1024 pixels (v4) raised the mAP50 from 0.363 to 0.454, mostly on logos and text. Re-annotating the dataset to 200 images (v5) raised it from 0.486 to 0.728; underlay recall rose from 0.10 to 0.52 and embellishment recall from 0.21 to 0.51. The v5 model, which had the highest mAP50 of the tested versions, was deployed.

Better annotations improved the detector more than a larger model did. The larger model helped on logos and text, which have clear boundaries; the re-annotation helped on underlays and embellishments, which had unclear ones. The expert's review agreed: only some class labels were wrong. dos Santos Júnior, Paixão, and Alvarez [2025] compared YOLO versions; in this study, the training data mattered more than the model. Consistent labeling therefore mattered most, and future datasets should start with clear class definitions. Panel A used different images from Panels B and C; future evaluations should compare every version on the same split to separate the effect of the model from that of the annotations. Most of the deployed detector's gain came from the annotations.

IoU was computed for Phase 1 only, between the deployed detector's boxes and the annotations of the 40-image validation split (Table 5). Each annotated box was paired with at most one detection, and a missed box counted as zero.

Table 5
Intersection over Union of the Deployed Detector on the 40 Validation Images

Class	Boxes	Mean IoU	Found	Precision	IoU of Found
Logo	140	0.844	0.957	0.957	0.882
Text	260	0.679	0.796	0.695	0.818
Underlay	41	0.548	0.634	0.605	0.850
Embellishment	166	0.445	0.530	0.633	0.809
All	**607**	**0.644**	**0.750**	**0.734**	**0.837**

"Found" (the recall) counts pairings with IoU ≥ 0.5; "IoU of Found" averages over those pairs only.

The mean IoU over the 607 annotated boxes was 0.644 (SD 0.351). The detector found 75.0% of the elements, with an average IoU of 0.837 (SD 0.097) for those found. Logos had the highest mean IoU (0.844, with 95.7% found), and embellishments the lowest (0.445). The mAP50 on the same images was 0.746.

The detector placed the elements it found accurately; its weakness was missing elements, mostly embellishments and underlays. The missed boxes, which counted as zero, explained both the gap between the mean IoU and the IoU of the found elements and the large SD: the boxes fell into two groups, missed ones at zero and found ones near 0.84. The mAP50 was close to the share of elements found, so a detected element was usually placed above the 0.5 IoU threshold, and its box could be used with little adjustment. Logos, which anchor brand consistency, were almost always found. As in the annotation process, the classes of Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] that are defined by appearance rather than content were the least consistent. The dashboard's controls for logos and markers covered what the detector missed. The 40 images came from a dataset of institutional and campus posters. The detector served logos and text well and the decorative classes less well.

Color Extraction and Stylistic Markers (K-Means Clustering and SIFT)

K-Means chose two to six colors by the Elbow Method, and SIFT measured the background's visual detail and ranked the candidate motifs. Table 6 presented how often the master's title artwork and motifs reached the outputs of the automatic Phase 3 battery.

Table 6
Transfer of Stylistic Markers in the Automatic Phase 3 Battery

Measure	Certificate	Event ID
Outputs with the master's title artwork	14 of 16	14 of 16
Outputs with decorative motifs	3 of 16	3 of 16
Motifs per output, when placed	2 to 4	2 to 4

Structure mapping + GNN outputs, before the per-poster choices made for the Phase 4 evaluation set.

An unseeded K-Means had given the same poster different palettes on identical runs, so every result in this chapter was produced after seeding. A review of the 32 outputs found three posters with a wrong title-artwork cut-out, one whose logotype was taken as a motif, and one whose "CLOSED" stamp was taken as a motif. A new rule fixed the logotype case; the others were corrected per poster for Phase 4, except on ANTI_HAZING_LAW.

K-Means and SIFT extracted the palette and markers automatically, but the transfer of artwork and motifs still needed human checking. The errors came from elements that look decorative but carry meaning. The CIE L*a*b* color space followed Garcia and Chen [2023], and seeding addressed the concern for stable centroids raised by Lertrusdachakul, Meesad, and Kittirachakul [2025]. SIFT keypoints are robust to scale [21], so they measured detail regardless of the master's size, but not meaning. The dashboard's tick boxes remained necessary. The palettes were not compared with designers' choices. Extraction worked, with human review as a safeguard.

Refinement of the Layout (Graph Neural Network)

The GNN was evaluated on 2,900 held-out CGL-Dataset layouts (Table 7). Position and gap errors were fractions of the canvas, and "order kept" was the share of consecutive element pairs in the designer's top-to-bottom order.

Table 7
GNN Accuracy on 2,900 Held-Out CGL-Dataset Layouts

Starting Layout → Model	**Position**	**Gap**	**Order Kept**	**Overlap**
Naive stack, no model	0.175	0.077	1.000	0.000
v1, as first deployed	0.210	0.349	0.472	0.002
Naive stack → v2	0.121	0.074	0.999	0.000
Naive stack → v3	0.121	0.074	0.999	0.000
Master's structure, no model	0.050	0.052	0.981	0.004
Master's structure → v2	0.096	0.097	0.924	0.004
Master's structure → v3 (deployed)	0.036	0.041	0.971	0.0005

The first version (v1) kept the order of 47.2% of pairs and had a higher position error than the naive stack (0.210 against 0.175). Trained from the naive stack, v2 reduced it to 0.121. The deployed v3, trained from the master's structure, reduced that structure's position error from 0.050 to 0.036, its gap error from 0.052 to 0.041, and its overlap by 89%; from the same start, v2 raised the error to 0.096. With v3, the share of pairs in order fell slightly, from 98.1% to 97.1%.

The GNN improved on the master's structure only when trained to start from it. A model must be trained from the kind of layout it receives; v3 learned small corrections to a structure already close. The layout models of Chapter 2 generated or represented layouts offline [11, 8], while this GNN refined a master's structure. A refining model should be trained from the structure it refines. It was tested on CGL-Dataset layouts with per-layout color and saliency features. Only v3 improved on the master's structure.

A second test measured how the GNN placed the elements without a counterpart on the master when the master's anchors were exact (Table 8).

Table 8
Vertical-Center Error With the Master's Exact Anchors (Fraction of the Height)

Method	Anchored	Unmapped	All
Linear interpolation (mapping only)	0.000	0.081	0.018
v3, used directly	0.026	0.084	0.040
v3, registered to the anchors (deployed)	0.000	0.075	0.017
v2, registered to the anchors	0.000	0.109	0.025

8,356 anchored and 2,432 unmapped elements in the held-out layouts.

Used directly, v3 moved the anchors by 2.6% of the height. Registered back onto the anchors, as in the deployed pipeline, it kept them exactly and placed the unmapped elements with a vertical-center error of 0.075, against 0.081 for linear interpolation (8% lower). Registered the same way, v2 had an error of 0.109. Over all elements, registration lowered v3's error from 0.040 to 0.017, just below interpolation (0.018). The unmapped elements were 2,432 of the 10,788 elements, about 23%.

Registration kept the master's positions exact and let the GNN place the remaining elements better than interpolation. Used alone, the GNN drifted from the anchors; registered, it kept them and still improved the other elements. The gain was small, which matched the small corrections seen in the first test, and it applied only to the unmapped elements, nearly a quarter of the total. Lee, Jiang, Essa, Le, Gong, Yang, and Yang [2020] generated graphic layouts under user-specified constraints; in ThemeSync, the master's anchors played that role. With registration, a learned model can work alongside positions that must not move. The 8% gain was measured on held-out CGL-Dataset layouts, and its effect on the final layouts was smaller, as the next subsection showed. Registration was the right way to combine the GNN with the master's anchors.

Mapping of Structural Dependencies

On the 16 unseen posters, the structure error measured how far the output's mapped elements (the logo row, header, title, and details) sat from their counterparts on the master, as a fraction of the height, averaged over the six candidates of each run (Table 9).

Table 9
Structure Error on the 16 Unseen Posters (Mean of Six Candidates per Run)

Format	Mapping + GNN	Mapping Only	Baseline
Academic Certificate (14 posters)	0.056	0.056	0.089
Event identification card (15 posters)	0.045	0.045	0.117

Posters without a mappable role for a format were excluded from that format's mean.

The structure mapping + GNN output was closer to the master than the baseline on every poster with mappable roles: 14 certificates and 15 event IDs. Two posters had no role that the certificate could take from the master, and one had none for the event ID; these were left out. On event IDs, the structure error fell from 0.117 to 0.045; on certificates, from 0.089 to 0.056. The structure mapping + GNN and structure mapping only conditions had the same structure error. The GNN moved the unmapped elements in 14 of the 32 runs, by at most 7.5% of the height.

The framework carried the master's structure into both target formats. The equal errors with and without the GNN showed that the transfer came from the mapping, while the GNN only adjusted the unmapped elements. It moved them little because both formats stack their elements vertically with little free space. Certificates gained less than event IDs because a certificate takes only the master's logo row and header. Wheeler [2013] treated consistency across applications as central to brand identity, and the transfer gave the target formats the master's arrangement as well as its colors. The structure error covered only the mapped elements and measured closeness to the master, not quality, which Objective 3 tested. The structure transfer worked as measured.

Figure 6 presented the structure error of each poster in both conditions.

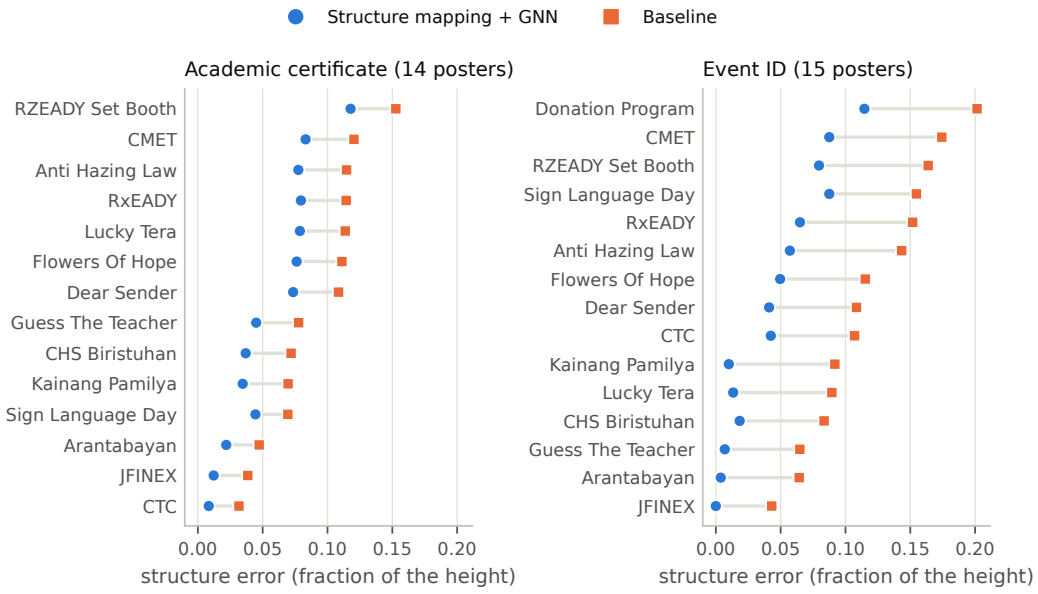


Figure 6: **Structure Error per Poster, Structure Mapping + GNN Against the Baseline**

Every poster with mappable roles had a lower structure error with structure mapping + GNN than with the baseline. The reduction was similar from poster to poster: 0.023 to 0.037 on certificates and 0.043 to 0.087 on event IDs. The remaining error varied more, from 0.008 to 0.118 on certificates and from 0.000 to 0.115 on event IDs.

The improvement was consistent across posters and was not driven by a few of them. The steady reduction showed that the mapping worked the same way on every master, while the remaining error depended on how far each master's arrangement was from what the target format's design rules allowed. Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] conditioned layout generation on the composition of the poster image; ThemeSync conditioned the target layout on the master's structure directly, and this held across varied masters. A designer could expect the master's arrangement to carry over for most event posters, not only typical ones. The 16 posters came from school organizations. Every mappable poster gained from the structure mapping.

Reactive Layout Synthesis Engine

The second specific objective was to develop a Reactive Layout Synthesis engine that used a Bi-directional State Machine and the Cassowary algorithm to solve Constraint Satisfaction Problems, propagating manual Hard Constraints in real time while preserving the global layout state. It was evaluated on the 64 rated layouts, on the synthetic templates filled with extreme payloads (52 runs), and on editing sessions timed on a workstation with an NVIDIA RTX 3060 GPU.

Constraint Satisfaction (Cassowary Algorithm)

Table 10 presented the F1-score and text fit of the 64 rated layouts, read from the printed files.

Table 10
Structural Accuracy and Text Fit of the 64 Rated Layouts

	Certificate		**Event ID**	
	Mapping + GNN	**Baseline**	**Mapping + GNN**	**Baseline**
F1-score	1.0 (all)	1.0 (all)	1.0 (all)	1.0 (all)
Reading order kept	16	16	16	16
All text within its box	16	16	14	14
Text over other content	0	0	0	0
Smallest text at print size	7.8 pt	7.8 pt	6.5 pt	6.5 pt

All 64 layouts reached an F1-score of 1.0, and none of their 384 candidates broke a constraint. On two event IDs, a header line was 6 to 19 pixels wider than its box, inside the card's panel. A crowded test edit lowered the F1-score to 0.90.

The solver kept every layout valid, so designers had no overlaps to fix. Cassowary satisfies required constraints and relaxes only preferences, as Badros, Borning, and Stuckey [2001] designed it to do. The F1-score checks boxes, not text, and was 1.0 in both conditions, so it could not separate them. Every rated layout was structurally valid.

The synthetic templates were filled with extreme payloads (Table 11): fields at 25% to 100% of their maximum length, one character over it, and 1 to 10 signature blocks.

Table 11
Stress Test of the Synthetic Templates

Condition	Academic Certificate	Event ID
Every field at its maximum (real words)	All text fit	The organization header overflowed
First failure (real words)	None	Organization header from 49 of 90 characters
First failure (one unbroken string)	Citation from 98 of 150 characters	Event details from 51 of 80 characters
Maximum + 1 character	Rejected (all 7 fields)	Rejected (all 5 fields)
Reading order	Held in every run	Held in every run
F1-score	1.0 in every length run	1.0 in every run
Signature blocks	Clean up to 5; overlapping from 6 (F1 0.92)	—

The boxes stayed apart, on the canvas, and in order in every run, and the schema rejected every over-length field. The certificate's name shrank from 31.3 to 13.5 points to fit 60 characters, while event ID header lines over 48 characters overflowed, and from about 56 characters they ran past the canvas edge. From six signature blocks, the blocks overlapped (F1-score 0.92), and from seven, the names spilled onto their neighbors. A long unbroken string spilled 3 to 48 pixels into the margin without covering another element. Overflowing runs still scored an F1-score of 1.0.

The solver held at every length; the limits were in text fitting, since the header's box did not shrink its text. Jiang, Stuerzlinger, Zwicker, and Lutteroth [2020] let adaptive layouts switch arrangements with OR-constraints, while ThemeSync's templates were fixed. Long organization names and more than five signatories need handling. The test covered the two synthetic templates. The engine was robust, with two text-fit limits.

Manual Hard Constraints (Bi-directional State Machine)

The Bi-directional State Machine recorded every terminal edit as a new, immutable state in the JSON state chain. Simulated moves, resizes, and locks were kept exactly through every later re-solve, with no overlap among the surrounding elements, and every re-solve of a session's chain was checked against the edits made before it. Undo and redo restored earlier states by appending them as new states, so the chain stayed append-only. Text edits were re-validated against the content schema before they were accepted. Each edit was made on the dashboard's canvas and produced exactly one terminal event, verified in a real browser. An edit entered the chain only when it was finished, on drop or at the end of a resize, so the positions during a drag were not recorded. The dashboard's Evaluation Report ran the same check on the user's own layout and listed each manual edit with whether it was still in place after the later re-solves.

The engine propagated manual Hard Constraints without overwriting earlier edits. A Hard Constraint was an element the designer had moved, resized, or locked; each became a required constraint and a new state, while the other elements stayed weighted preferences, so a re-solve could move only the elements the designer had not fixed. Generative layout models recalculate the whole layout and can overwrite manual changes [22]; the state chain instead kept every edit, in line with the human-in-the-loop co-design that He, Huang, and Wang [2022] demonstrated with iPLAN and with the bi-directional state machine of Liang, McGee, Menassa, and Kamat [2020]. This addressed the destructive state management identified in Chapter 1. Because undo and redo appended states instead of deleting them, a designer could return to any earlier version without losing the history, and the dashboard's State Timeline showed that history in full. The checks used simulated sessions, not designers' own sessions, so how designers edit in practice was not tested. The engine met the requirement to preserve manual edits.

Real-Time Propagation (Algorithmic Latency)

Latency was measured over 34 editing sessions of 40 edits each (1,360 edits) on the 16 posters in both formats and on the synthetic theme (Table 12). The solver timing was the constraint recalculation, and the full edit was the state synchronization.

Table 12
Latency of the Designer's Edits

Timing	Edits	Median	95th Pct.	Max	$\leq$ 0.1 s	$\leq$ 1.0 s
Solver	1,012	0.4 ms	0.8 ms	95.9 ms[a]	100%	100%
Full edit	1,360	90 ms	176 ms	206 ms	61%	100%
Certificate	680	146 ms	182 ms	206 ms	29%	100%
Event ID	680	86 ms	103 ms	159 ms	93%	100%

Solver: the constraint recalculation. Full edit: the state synchronization (re-solve, re-score, re-draw, and save). [a]One solve paused by the operating system; every other solve took at most 1.9 ms. The browser's redraw of the page is not included. Means: solver 0.53 ms (SD 3.01, raised by the paused solve); full edit 90.6 ms (SD 57.5).

Every recalculation finished within 0.1 seconds, and all but one, paused by the operating system, within 1.9 milliseconds. Every full edit finished within 1.0 second, and 61% within 0.1 seconds. Full edits took longer on certificates than on event IDs (median 146 against 86 milliseconds).

The solver responded instantly, and every full edit stayed within the limit for uninterrupted work. Most of a full edit's time went to re-scoring legibility against the poster's background, and certificates have more elements and a larger canvas. Against the limits of Nielsen [1993], the solver was instantaneous and the full edit did not interrupt the task, in line with the interactive purpose of Cassowary [1] and the real-time constraint-based layout of Li, Li, Fu, and Cai [2024]. A designer could edit without waiting. Latency was measured on one workstation, without the browser's redraw. The engine met the real-time requirement.

Figure 7 presented the cumulative share of edits by time, with the 0.1-second and 1.0-second limits.

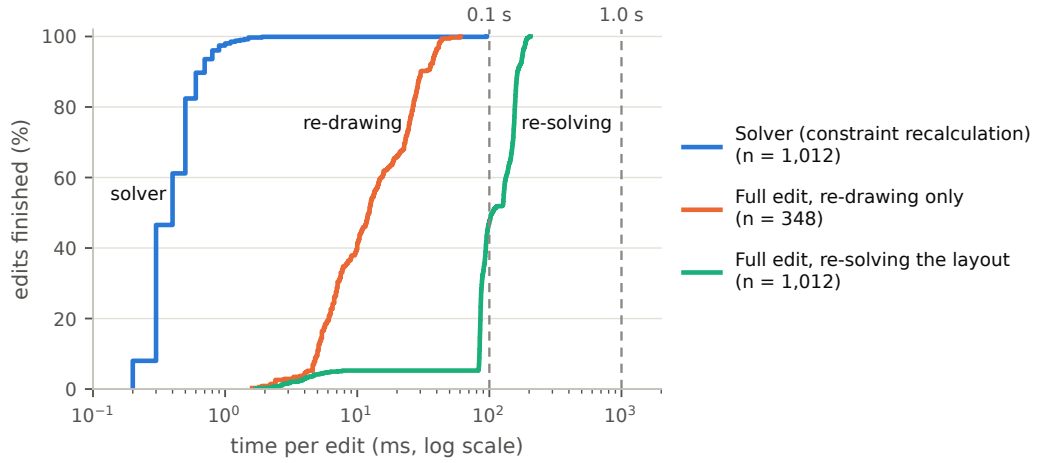


Figure 7: **Cumulative Share of Edits by Time, With the 0.1-Second and 1.0-Second Limits**

Edits that only re-drew the canvas (text edits, typography changes, undo, redo, restore, and candidate selections) all finished within 0.1 seconds, with a median of 12 milliseconds. Edits that re-solved the layout (moves, resizes, and locks) had a median of 105 milliseconds, and 48% of them finished within 0.1 seconds. Opening a layout, which ran every stage of the pipeline, took a median of 3.4 seconds. The timings included an image cache that left the layouts byte-identical.

The edits fell into two groups: re-draws, which were instant, and re-solves, which met the 0.1-second limit about half the time. A full edit on the synthetic theme took about 4 milliseconds, so re-scoring legibility against the poster's background, not the solver, set the time of the re-solves. Nielsen [1993] placed the limit for a response that feels instantaneous at 0.1 seconds; about half of the re-solves fell just above it. Cheaper legibility scoring would bring most of them under 0.1 seconds. Opening a layout took seconds, but only once per layout. Real-time editing was achieved, with room to make re-solves faster.

Heuristic Evaluation and Expert Validation

The third specific objective was to evaluate the generated layouts with a mathematically calibrated Heuristic Fitness Function that ranked parallel design candidates, followed by a quantitative human-expert validation. It was evaluated on the Phase 3 battery (96 runs), run on the school's server with an NVIDIA RTX A5000 GPU, and on the Phase 4 ratings of four professional designers.

Ranking of Parallel Design Candidates (Heuristic Fitness Function)

Table 13 presented the fitness of the output layouts, each chosen from six candidates solved by Cassowary and scored by Formula 3.11.

Table 13
Heuristic Fitness of the Output Layouts on the 16 Unseen Posters (Means)

Format	**Condition**	F_{layout}	$P_{whitespace}$	$P_{legibility}$
Certificate	Mapping + GNN	0.0125	0.0108	0.0011
	Baseline	0.0126	0.0108	0.0012
Event ID	Mapping + GNN	0.1633	0.1508	0.0083
	Baseline	0.1583	0.1508	0.0050

Lower is better. $P_{collision} = 0$ and $P_{alignment} \approx 0$ in every run. SD of F_{layout} (mapping + GNN, baseline): certificate 0.0250, 0.0251; event ID 0.0133, 0.0135.

The two conditions scored the same on 10 of the 16 certificates, and the baseline scored lower (better) on event IDs through the legibility penalty. Over the 32 pairs, the Wilcoxon signed-rank test favored the baseline ($p = 0.030$, $r = +0.53$).

The fitness function slightly favored the baseline. The legibility term penalized text placed over the poster's artwork, while the whitespace term was equal in both conditions. Like the energy model of O'Donovan, Agarwala, and Hertzmann [2014], Formula 3.11 combined penalties for overlap, whitespace, and misalignment, but none of its terms measured where the space went or how closely the output followed the master. The fitness function measured little of what separated the conditions.

In an earlier run in which the fitness function alone chose the output, “classic centered” won 14 of the 16 certificates because the top candidates tied exactly, and “left-aligned” won all 16 event IDs by a small, steady margin. The master’s composition, read from the detected elements, then chose the archetype, and the fitness function ranked the candidates within it (Table 14).

Table 14
Layout Archetypes Selected From the Masters’ Compositions (16 Posters)

Certificate		Event ID	
Classic centered	11	Classic centered	9
Header banner	2	Floating card	2
Art column	2	Full-width band	2
Left-aligned	1	Left-aligned	3

The 16 posters received four archetypes in each format, and both conditions received the same archetype on every poster. The classic centered layout was the most common in both formats (11 certificates and 9 event IDs). The chosen candidate did not always have the lowest fitness: the art column scored about 0.076 against 0.003 for the classic layout, and the floating card about 0.187 against 0.149 for the left-aligned layout.

The master’s composition, not the fitness function, gave the outputs their variety. The fitness function could not tell the archetypes apart well, since its top candidates tied or differed by small, constant margins. Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] conditioned layout generation on the composition of the poster image; ThemeSync applied the same idea by taking the archetype from the master’s composition. A fitness function alone was not enough to choose among layout types. Choosing by composition meant accepting some candidates with a higher fitness, and the rules that read the archetype from the master were set by the researchers without a separate validation. Each poster received a layout type that followed its master.

Mathematical Calibration of the Fitness Weights

The weights of Formula 3.11 were set by the priority rule of Chapter 3 and calibrated on the 64 rated runs with criteria fixed in advance: a change to one weight (by a factor of 0.25 to 4) was kept only if the selected layouts improved on a design rule and worsened on none. Table 15 presented how many selections changed when one weight was scaled.

Table 15
Selected Layouts That Changed When One Fitness Weight Was Scaled (of 64)

Weight (Deployed Value)	$\times 0$	$\times 0.25$	$\times 0.5$	$\times 2$	$\times 4$
α, collision (50)	0	0	0	0	0
β, alignment (2)	0	0	0	0	0
γ, whitespace (1)	11	1	1	1	0
δ, legibility (1.5)	5	1	1	0	0

$\times 0$ removed the term; the other weights stayed at their deployed values. Every setting kept the constraints in all 64 runs.

The loop kept no change, which confirmed the deployed weights. Halving or doubling any weight changed at most 1 of the 64 selections; removing the whitespace or legibility term changed 11 or 5. Of the 48 runs with two candidates of the chosen archetype, 22 tied exactly. A whitespace target taken from each master instead of 0.65 would have changed 2 layouts. Every setting kept the constraints in all 64 runs, and α stayed the highest weight unless it was removed. The calibration ran before any rating was analyzed.

The weights were stable. They mattered little because no candidate overlapped, the alignment penalty was near zero, and candidates of one archetype had nearly the same whitespace. O'Donovan, Agarwala, and Hertzmann [2014] learned the weights of their energy terms from example designs; here, the weights were not the deciding factor. Improving the fitness function would need new terms, not new weights. The calibration used the formula's own terms, not the experts' ratings. The weights were confirmed and robust.

Human-Expert Validation (Mean Opinion Score)

Four professional designers rated the 64 layouts on the three parameters of Table 2 without knowing the conditions (768 ratings). The rated pages of ANTI_HAZING_LAW carried two extraction defects in both conditions. The analysis followed the statistical treatment of Chapter 3, fixed in advance (Table 16).

Table 16
Mean Opinion Score by Format, Condition, and Parameter

Format	Condition	Hierarchy	Brand	Whitespace	Overall
Certificate	Mapping + GNN	3.88 ± 1.05	3.56 ± 1.02	3.06 ± 1.28	3.50 ± 0.89
	Baseline	3.73 ± 1.20	3.78 ± 1.00	3.81 ± 1.19	3.78 ± 0.98
Event ID	Mapping + GNN	3.34 ± 1.01	3.38 ± 1.15	2.78 ± 1.06	3.17 ± 0.85
	Baseline	3.72 ± 1.02	3.62 ± 1.05	3.73 ± 1.07	3.69 ± 0.87
Both	Mapping + GNN	3.61 ± 1.06	3.47 ± 1.09	2.92 ± 1.18	3.33 ± 0.89
	Baseline	3.73 ± 1.11	3.70 ± 1.02	3.77 ± 1.13	3.73 ± 0.93

Mean ± SD of 64 ratings per cell for one format and 128 for both. The overall score is the mean of the three parameters.

The overall MOS was 3.53 (SD 0.93), between Acceptable and Good. The baseline was rated higher on every parameter and overall (3.73 against 3.33), most on Constraint and Whitespace Harmony (3.77 against 2.92) and least on Visual Hierarchy Preservation (3.73 against 3.61). The difference was larger on event IDs (3.69 against 3.17) than on certificates (3.78 against 3.50). The one exception was Visual Hierarchy Preservation on certificates (3.88 for structure mapping + GNN against 3.73).

The experts preferred the baseline, mostly for its use of space; keeping the master's positions did not make the hierarchy clearer. O'Donovan, Agarwala, and Hertzmann [2014] called white space "fundamental for readability and aesthetics," and the ratings reflected this. Following a master's structure did not by itself earn higher ratings. The MOS came from 16 posters and four experts. The baseline was preferred overall.

Figure 8 presented how often each rating was given, by parameter and condition.

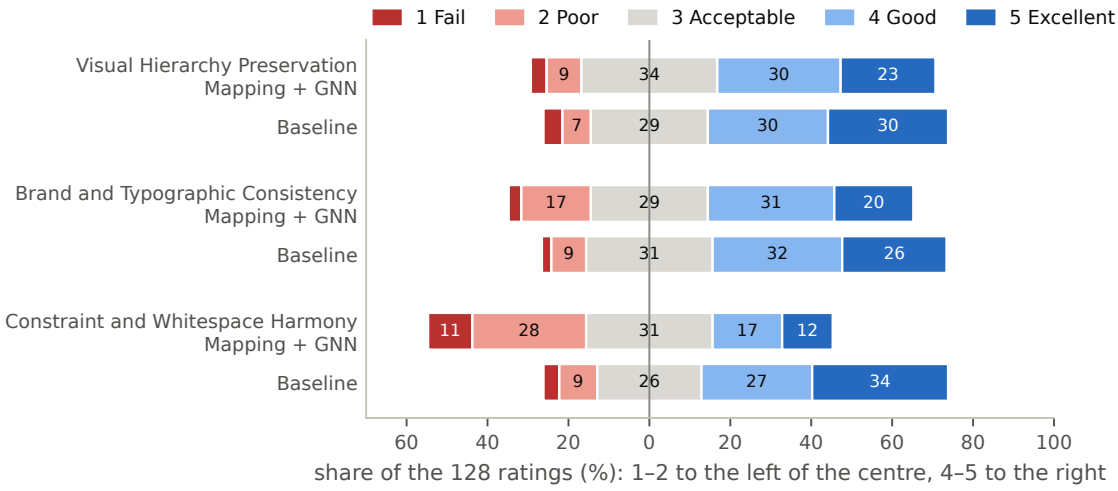


Figure 8: **Distribution of the Ratings by Parameter and Condition**

Across the three parameters, structure mapping + GNN received 92 ratings of 1 or 2 and 71 ratings of 5, out of 384. The baseline received 46 ratings of 1 or 2 and 114 ratings of 5. On Constraint and Whitespace Harmony alone, structure mapping + GNN received 50 ratings of 1 or 2, against 17 for the baseline.

Structure mapping + GNN received twice as many low ratings as the baseline and fewer top ratings. Most of the extra low ratings were on Constraint and Whitespace Harmony, so a specific weakness, not a general one, drove the difference. Patnaik, Jain, Krishnamurthy, and Sarkar [2025] aligned layout generation with human aesthetic preferences, and the distribution showed why such preferences have to be measured directly: the objective checks rated both conditions as equally valid. Fixing the spacing would address most of the low ratings. The counts pooled four experts who differed in leniency. The low ratings concentrated on whitespace.

Table 17 presented the agreement of the four experts on the order of the 64 layouts.

Table 17
Agreement of the Four Experts Over the 64 Layouts (Kendall's W)

Parameter	W	χ^2 **(df = 63)**	p
Visual Hierarchy Preservation	0.579	145.9	< 0.001
Brand and Typographic Consistency	0.477	120.2	< 0.001
Constraint and Whitespace Harmony	0.570	143.5	< 0.001
Overall	**0.630**	**158.7**	< 0.001

On the scale of Schmidt [1997], Kendall's W showed moderate agreement overall (0.630), on Visual Hierarchy Preservation (0.579), and on Constraint and Whitespace Harmony (0.570), and weak agreement on Brand and Typographic Consistency (0.477). All four values were significant ($p < 0.001$). As a secondary check, the absolute agreement was lower: ICC(2,1) ranged from 0.16 to 0.31 and Krippendorff's α from 0.01 to 0.23. The experts' individual mean ratings ranged from 2.88 to 4.32, a spread of 1.44 points on the five-point scale.

The experts ranked the layouts similarly but used the scale differently. Kendall's W measures agreement on order, while the ICC and Krippendorff's α measure agreement on the scores themselves, so the experts agreed on which layouts were better while some rated everything higher than others. Nielsen and Molich [1990] found that single evaluators miss problems that several evaluators together find, which supported pooling the four experts; by the guidelines of Koo and Li [2016] and of Krippendorff [2004], however, their absolute agreement was poor. The comparison between the conditions, which relied on the order within pairs, was therefore more dependable than the absolute MOS values. Only four experts rated, with no calibration round before the rating to align their use of the scale. The agreement was sufficient for comparing the conditions.

Table 18 presented the Wilcoxon signed-rank tests on the 32 pairs, at a significance level of 0.05.

Table 18
Wilcoxon Signed-Rank Tests on the 32 Pairs

Parameter	**B > M**	**M > B**	**Equal**	**Median Diff.**	T	p_{Holm}	r
Overall (main test)	25	2	5	-0.42	9.5	< 0.001	-0.95
Hierarchy	16	7	9	-0.13	88.5	0.127	-0.36
Brand	22	5	5	-0.25	50.0	< 0.001	-0.74
Whitespace	26	2	4	-1.00	4.0	< 0.001	-0.98

B > M, M > B: pairs rated higher for the baseline or for structure mapping + GNN. A negative r favors the baseline. Two-sided tests, Holm-corrected over the four tests.

The experts rated the baseline significantly higher overall, the main test ($T = 9.5$, $p_{Holm} < 0.001$, $r = -0.95$, a large effect), and on Brand and Typographic Consistency and Constraint and Whitespace Harmony. Visual Hierarchy Preservation showed no significant difference ($p_{Holm} = 0.127$). The paired t-test and the Bonferroni correction agreed. Without the one pair whose layouts rendered identically, Visual Hierarchy Preservation reached significance ($p_{Holm} = 0.046$), so that result was borderline.

The preference for the baseline was significant, large, and robust, the opposite of the expected direction. Because the GNN's output was registered to the master's anchors, the difference reflected mainly the structure mapping. Zhou, Xu, Ma, Ge, Jiang, and Xu [2022] had designers judge generated posters alongside objective measures; here, the F1-score was equal in both conditions, yet the experts separated them, which supported human evaluation as practiced by Li, Liu, Feng, Zhu, Li, Zhang, Lv, Zhu, Shen, Lin, and Shao [2023]. The structure mapping needs a step that evens out the spacing. The baseline lacked both the mapping and the GNN, so the test could not isolate the GNN. The expert validation did not support the structure mapping + GNN condition.

Figure 9 presented the overall MOS of each of the 32 pairs.

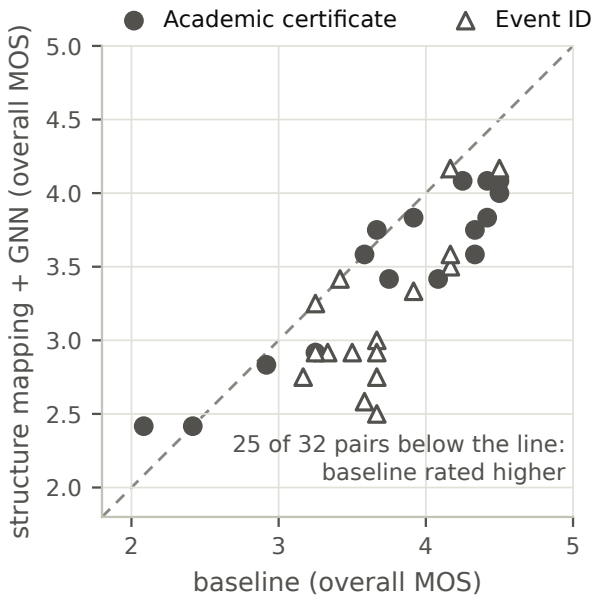


Figure 9: **Overall Mean Opinion Score of the 32 Pairs**

Of the 32 pairs, 25 lay below the line of equal scores, in favor of the baseline. In exploratory analyses, which were added after the planned tests and not corrected, each of the four experts rated the baseline higher on average, by 0.15 to 0.75 points. Without ANTI_HAZING_LAW, whose rated pages carried two defects, the conclusions were unchanged (overall $p_{Holm} < 0.001$, $r = -0.95$; Visual Hierarchy Preservation $p = 0.153$).

The preference for the baseline was spread across the pairs and the experts. It did not depend on a few posters, on one expert, or on the defective poster. As in the method of ITU-T P.800 [6], the MOS averaged several raters, and the per-expert results showed that the average was not driven by any one of them. The result can be taken as a consistent judgment of these experts, not an artifact of one rater or poster. The per-expert and exclusion analyses were exploratory. The preference for the baseline was consistent.

A post hoc comparison measured the spacing of the rated layouts (Table 19). The largest gap and the stack spread were fractions of the layout's height, and the gap CV was the coefficient of variation of the gaps.

Table 19
Spacing of the Rated Layouts (Post Hoc)

Measure	Mapping + GNN	Baseline	ρ	p
Largest gap	0.072 ± 0.057	0.020 ± 0.003	-0.42	0.017
Gap CV	0.47 ± 0.44	0.00 ± 0.00	-0.42	0.017
Stack spread	0.816 ± 0.054	0.746 ± 0.025	-0.45	0.010

Mean $\pm$ SD over the 32 layouts of each condition. ρ: Spearman correlation over the 32 pairs between the change in the measure (structure mapping + GNN minus baseline) and the change in overall MOS; exploratory and uncorrected.

Structure mapping + GNN spread the elements over more of the height (0.816 against 0.746) and left larger, uneven gaps (largest gap 0.072 against 0.020; gap CV 0.47 against 0.00). The baseline stacked the elements with even gaps. The pairs in which the gaps grew the most lost the most MOS (Spearman's ρ from -0.42 to -0.45, p from 0.010 to 0.017).

Uneven spacing was the most likely reason for the lower ratings. Structure mapping placed the logo row, header, title, and details at their relative positions on the master, and because a poster has different proportions from a certificate or an event ID, these positions opened large gaps in the target, such as an empty band between the header and the title. O'Donovan, Agarwala, and Hertzmann [2014] penalized whitespace imbalance in their energy model, whereas the whitespace term of Formula 3.11 measured only the amount of empty space. The anchored positions should be balanced with an even distribution of the remaining space, for example by redistributing the gaps after anchoring. This analysis was exploratory and correlational, so it did not prove that spacing caused the lower ratings. Spacing explained the difference better than any measure the system used.

Figure 10 presented the change in the largest gap against the change in overall MOS for the 32 pairs.

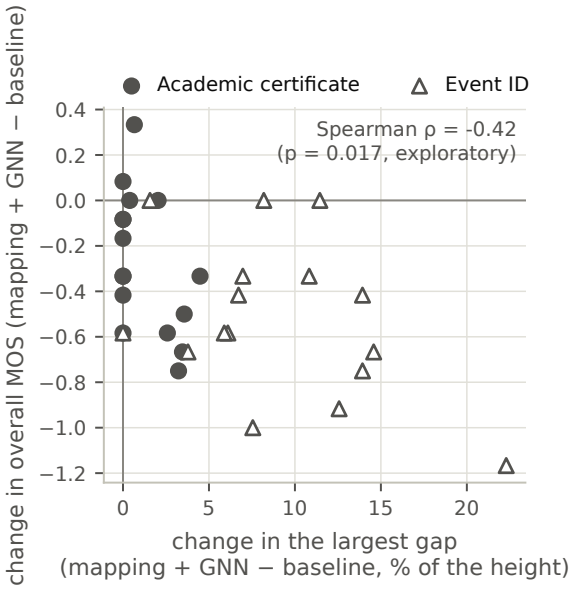


Figure 10: **Change in the Largest Gap Against the Change in Overall MOS (Exploratory)**

The largest gap of structure mapping + GNN was larger than the baseline's in 23 of the 32 pairs and smaller in none. Larger increases in the gap went with larger losses of MOS. The difference in fitness between the two conditions did not follow the difference in MOS ($\rho = -0.20$, $p = 0.26$).

The experts' ratings followed the spacing, and the fitness function did not. The whitespace term of Formula 3.11 compares the ratio of empty to filled area, which is nearly the same for even and uneven gaps, so the fitness function could not see the change the experts reacted to. Guo, Fu, and Zhu [2025] turned design rules such as whitespace and visual density into objective metrics; the present result showed that such a metric has to capture how the space is distributed, not only how much of it there is. A term for the evenness of the gaps should be added to the fitness function. The figure showed a correlation from an exploratory analysis. The fitness function missed the spacing that the experts judged.

Notes

[1] Greg J. Badros, Alan Borning, and Peter J. Stuckey. 2001. The Cassowary linear arithmetic constraint solving algorithm. *ACM Transactions on Computer-Human Interaction*, 8, 4, 267–306. DOI: 10.1145/504704.504705.

[2] Eder Silva dos Santos Júnior, Thuanne Paixão, and Ana Beatriz Alvarez. 2025. Comparative performance of yolov8, yolov9, yolov10, and yolov11 for layout analysis of historical documents images. *Applied Sciences*, 15, 6, 3164. DOI: 10.3390/app15063164.

[3] Miguel Garcia and Jia-Ray Chen. 2023. New approach to dominant color extraction in images with wide range of hues using CIE lab space. *Technologies*, 11, 6, 230. DOI: 10.3390/technologies11060230.

[4] Xiaocen Guo, Sijia Fu, and Di Zhu. 2025. Aesthetic quality evaluation of packaging design with graph neural networks and composition features. *Scientific Reports*, 15, 1, 36046. DOI: 10.1038/s41598-025-20046-1.

[5] Feixiang He, Yanlong Huang, and He Wang. 2022. Iplan: interactive and procedural layout planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7783–7792. DOI: 10.1109/CVPR52688.2022.00764.

[6] ITU-T. 1996. Methods for subjective determination of transmission quality. ITU-T Recommendation P.800. International Telecommunication Union, Geneva, Switzerland. Retrieved October 12, 2026 from https://www.itu.int/rec/t-rec-p.800.

[7] Yue Jiang, Wolfgang Stuerzlinger, Matthias Zwicker, and Christof Lutteroth. 2020. ORCSolver: an efficient solver for adaptive GUI layout with OR-constraints. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '20)*. ACM, 1–14. DOI: 10.1145/3313831.3376610.

[8] Yue Jiang, Changkong Zhou, Vikas Garg, and Antti Oulasvirta. 2024. Graph4gui: graph neural networks for representing graphical user interfaces. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. Retrieved October 9, 2026 from https://arxiv.org/abs/2404.13521.

[9] Terry K. Koo and Mae Y. Li. 2016. A guideline of selecting and reporting intraclass correlation coefficients for reliability research. *Journal of Chiropractic Medicine*, 15, 2, 155–163. DOI: 10.1016/j.jcm.2016.02.012.

[10] Klaus Krippendorff. 2004. *Content Analysis: An Introduction to Its Methodology*. (2nd ed.). Sage Publications, Thousand Oaks, CA, USA. Retrieved October 12, 2026 from https://books.google.com/books/about/Content_Analysis.html?id=q657o3M3C8cC.

[11] Hsin-Ying Lee, Lu Jiang, Irfan Essa, Phuong B. Le, Haifeng Gong, Ming-Hsuan Yang, and Weilong Yang. 2020. Neural design network: graphic layout generation with constraints. In *Proceedings of the European Conference on Computer Vision (ECCV '20)*. Retrieved October 9, 2026 from https://arxiv.org/abs/1912.09421.

[12] Tanakorn Lertrusdachakul, Phayung Meesad, and Sunantha Kittirachakul. 2025. Enhancing clustering performance: a hybrid generalized K-means approach. *International Journal of Advanced Research (IJAR)*, 13, 1. Retrieved March 3, 2026 from https://www.journalijar.com/uploads/2025/04/6821b3f719224_IJAR-50827.pdf.

[13] Bohan Li, Dawei Li, Ming Fu, and Shaowei Cai. 2024. SMT-layout: a MaxSMT-based approach supporting real-time interaction of real-world GUI layout. *arXiv preprint arXiv:2411.12271*. Retrieved March 3, 2026 from https://arxiv.org/abs/2411.12271.

[14] Fengheng Li, An Liu, Wei Feng, Honghe Zhu, Yaoyu Li, Zheng Zhang, Jingjing Lv, Xin Zhu, Junjie Shen, Zhangang Lin, and Jingping Shao. 2023. Relation-aware diffusion model for controllable poster layout generation. In *Proceedings of the 32nd*

ACM International Conference on Information and Knowledge Management (CIKM '23). Association for Computing Machinery, New York, NY, USA, 1249–1258. DOI: 10.1145/3583780.3615028.

[15] Cheng Liang, Wesley McGee, Carol Menassa, and Vineet Kamat. 2020. Bi-directional communication bridge for state synchronization between digital twin simulations and physical construction robots. In *Proceedings of the International Symposium on Automation and Robotics in Construction (ISARC 2020)*. DOI: 10.22260/ISARC2020/0205.

[16] Jakob Nielsen. 1993. *Usability Engineering*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 135. Retrieved October 12, 2026 from https://archive.org/details/usabilityenginee0000niel.

[17] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)*. Association for Computing Machinery, New York, NY, USA, 249–256. DOI: 10.1145/97243.97281.

[18] Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. 2014. Learning layouts for single-page graphic designs. *IEEE Transactions on Visualization and Computer Graphics*, 20, 8, 1200–1213. DOI: 10.1109/TVCG.2014.48.

[19] Sohan Patnaik, Rishabh Jain, Balaji Krishnamurthy, and Mausoom Sarkar. 2025. Aesthetiq: enhancing graphic layout design via aesthetic-aware preference alignment of multi-modal large language models. In *Proceedings of the CVPR '25*. IEEE, 23701–23711. DOI: 10.1109/CVPR52734.2025.02207.

[20] Roy C. Schmidt. 1997. Managing Delphi surveys using nonparametric statistical techniques. *Decision Sciences*, 28, 3, 763–774. Retrieved October 12, 2026 from https://doi.org/10.1111/j.1540-5915.1997.tb01330.x.

[21] M. Shaharom and K. Tahar. 2023. Multispectral image matching using SIFT and SURF algorithm: a review. *International Journal of Geoinformatics*, 19, 1, 13–21. Retrieved March 3, 2026 from https://ijg.e-geoinfo.com/index.php/journal/article/view/2495/1329.

[22] Yong Shi, Mengyuan Shang, and Zhiquan Qi. 2023. Intelligent layout generation based on deep generative models: a comprehensive survey. *Information Fusion*, 100, 101940. DOI: 10.1016/j.inffus.2023.101940.

[23] Alina Wheeler. 2013. *Designing Brand Identity: An Essential Guide for the Whole Branding Team*. (4th ed.). John Wiley & Sons, Hoboken, NJ, USA, 29. Retrieved October 12, 2026 from https://books.google.com/books/about/Designing_Brand_Identity.html?id=uSNHAAAAQBAJ.

[24] Min Zhou, Chenchen Xu, Ye Ma, Tiezheng Ge, Yuning Jiang, and Weiwei Xu. 2022. Composition-aware graphic layout gan for visual-textual presentation designs. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*. International Joint Conferences on Artificial Intelligence Organization, 4995–5001. DOI: 10.24963/ijcai.2022/692.

CHAPTER 5

SUMMARY OF FINDINGS, CONCLUSIONS, AND RECOMMENDATIONS

This chapter presented the summary of the study, its findings, the conclusions drawn from the findings, and the recommendations drawn from the conclusions.

Summary

The study developed and evaluated ThemeSync, a graph-based design orchestration framework that transferred the visual identity of a master layout, such as an event poster, to new formats while the designer kept control. The general objective was to design and evaluate a framework with real-time constraint satisfaction and stateful manual refinement across formats. The specific objectives were to design a Relational Logic and Feature Extraction model, to develop a Reactive Layout Synthesis engine, and to evaluate the generated layouts with a calibrated Heuristic Fitness Function followed by a human-expert validation. It covered two-dimensional layouts in two formats, the Academic Certificate and the event identification card, and the user kept the final design authority.

The study used a Constructive Research Design in four phases. Phase 1 extracted the master's palette with K-Means clustering, its stylistic markers with SIFT, and its elements with a YOLOv8s detector trained on 200 annotated master layouts, and a Graph Neural Network (GNN) trained on the CGL-Dataset V2 refined the target layout from the master's structure. In Phase 2, the Cassowary solver arranged the elements, and a Bi-directional State Machine recorded each manual edit as a Hard Constraint in a JSON state chain. Phase 3 generated six candidates per layout and ranked them with the Heuristic Fitness Function (Formula 3.11). Phase 4 was the expert validation.

The respondents were four professional designers. They rated 64 layouts made from 16 event posters chosen by purposive sampling, in two formats and two conditions: structure mapping + GNN, and a baseline that used neither. The ratings were gathered blind, in printed packets, on three parameters and a five-point scale adapted from ITU-T P.800. The system's own metrics were logged automatically: Intersection over Union (IoU), the GNN's errors, the F1-score, edit latency, and fitness.

The statistical treatment was fixed before the ratings were examined. It reported the Mean Opinion Score (MOS) with its standard deviation, measured agreement with Kendall's W, and compared the conditions with the two-sided Wilcoxon signed-rank test on 32 pairs, with the Holm correction, the rank-biserial correlation, and a significance level of 0.05.

The framework transferred the structure of real master layouts, kept every rated layout valid, and re-solved layouts in real time without losing manual edits, but the experts rated the baseline significantly higher than structure mapping + GNN, mainly for its spacing.

Findings

Each finding corresponded to one specific objective.

1. The detector trained only on the CGL-Dataset made no detection at the 0.25 threshold on three real event posters. The deployed detector, trained on 200 annotated master layouts, reached an mAP50 of 0.728 and a mean IoU of 0.644, finding 75.0% of the annotated elements. In the Phase 3 battery, the title artwork reached 14 of the 16 outputs of each format, and a review found five extraction errors. On 2,900 held-out layouts, the deployed GNN reduced the master structure's position error from 0.050 to 0.036. On the 16 unseen posters, structure mapping + GNN reduced the structure error from 0.117 to 0.045 on event IDs and from 0.089 to 0.056 on certificates, and the GNN moved the unmapped elements by at most 7.5% of the height.

2. All 64 rated layouts had an F1-score of 1.0, and manual moves, resizes, and locks were kept through every later re-solve. Every constraint recalculation finished within 0.1 seconds (median 0.4 milliseconds), and every full edit within 1.0 second (median 90 milliseconds); 48% of the edits that re-solved the layout finished within 0.1 seconds. In the stress test, the F1-score stayed at 1.0 at every text length, but the event ID's organization header overflowed its box beyond 48 characters per line, and the certificate's signature blocks overlapped from six blocks.

3. The baseline had a slightly better fitness than structure mapping + GNN (0.0855 against 0.0879; $p = 0.030$), and halving or doubling any fitness weight changed at most 1 of the 64 selected layouts. The experts rated the baseline significantly higher overall (MOS 3.73 against 3.33; $p_{Holm} < 0.001$, $r = -0.95$), most of all on Constraint and Whitespace Harmony (3.77 against 2.92), and they agreed moderately on the order of the layouts (Kendall's $W = 0.630$). In the exploratory analysis, structure mapping + GNN left a larger largest gap than the baseline in 23 of the 32 pairs, the pairs whose gaps grew more lost more MOS, and the difference in fitness did not follow the difference in MOS.

Conclusions

Each conclusion was drawn from the finding with the same number.

1. The first objective was met. The Relational Logic and Feature Extraction model identified the structure and stylistic markers of real master layouts and carried that structure into new formats. The detector worked only after training on in-domain data, the transfer of artwork and motifs still needed human review, and the GNN improved the master's structure but changed the final layouts little.

2. The second objective was met. The Reactive Layout Synthesis engine kept every layout structurally valid, kept every manual edit, and responded in real time. Its limits were in text fit, which the F1-score missed, and in the time of re-solving edits.

3. The third objective was carried out, but the expert validation did not support the structure mapping + GNN condition. The experts preferred the evenly spaced baseline. The Heuristic Fitness Function missed the uneven spacing they penalized, which the largest gap reflected better, and the weights were unlikely to be the cause, since changing them hardly changed the selected layouts. Following the master's structure did not by itself produce better layouts in the target formats.

Recommendations

Each recommendation was drawn from the conclusion with the same number.

1. Expand the empirical dataset with more varied master layouts and clearer class definitions, so that the detector and the marker transfer need less correction. Give the GNN per-element color and saliency features, and test it on formats with more free space.

2. Fit the event ID's organization header to the width of its box, limit the certificate to five signature blocks, and measure text fit together with the F1-score. Make the legibility scoring cheaper so that more re-solving edits finish within 0.1 seconds.

3. Distribute the remaining space evenly after anchoring the master's positions, and add a term for the evenness of the gaps to the Heuristic Fitness Function. Then compare the structure mapping alone, the structure mapping with the GNN, and the baseline in a new expert study with more evaluators.

APPENDICES

APPENDIX A

PREPROCESSED CGL-DATASET NODE ARRAY FOR GNN INGESTION


```
{
  "extracted_nodes": [
    {
      "semantic_category": "logo",
      "bbox": [106, 22, 284, 65]
    },
    {
      "semantic_category": "text",
      "bbox": [61, 140, 393, 67]
    }
  ],
  "extracted_color_centroids": [[135, 192, 167], [255, 128, 128]],
  "extracted_saliency_weight": 0.45
}
```

APPENDIX B

DYNAMIC CONTENT PAYLOAD FOR REAL-TIME RUNTIME ORCHESTRATION

```
{
    "domain_context": "academic_certificate",
    "target_aspect_ratio": "4:3",
    "variable_schema": {
        "{{Institution_Logo}}": {
            "structural_class": "image_node",
            "spatial_hierarchy": 0,
            "lock_aspect_ratio": true
        },
        "{{Issuing_Organization}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 1,
            "max_characters": 60
        },
        "{{Certificate_Title}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 2,
            "max_characters": 40
        },
        "{{Presentation_Line}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 3,
            "max_characters": 30
        },
        "{{Student_Name}}": {
```

```
            "structural_class": "text_node",
            "spatial_hierarchy": 4,
            "max_characters": 60
        },
        "{{Citation_Body}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 5,
            "max_characters": 150
        },
        "{{Date_Issued}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 6,
            "max_characters": 30
        },
        "{{Certificate_ID}}": {
            "structural_class": "text_node",
            "spatial_hierarchy": 7,
            "max_characters": 20
        },
        "{{Signature_Blocks}}": {
            "structural_class": "repeating_group",
            "spatial_hierarchy": 8,
            "instantiation_count": 3
        }
    }
}
```